# AI Native Workflow 5.6

Detailed Rationale, Hierarchical Execution Architecture, Human-Attention-Aware Oversight, and Evidence Control

Yuhe Sui

3 September 2026
Version 5.6.39

**Abstract**

Researchers increasingly compose frontier AI systems informally: a persistent ChatGPT- or Claude-style session may plan and interpret a project, a research surface may search literature, and Codex- or Claude-Code-style agents may implement methods and run experiments. The difficulty is no longer obtaining a capable individual response. It is sustaining a coherent research programme across many decisions, artifacts, model contexts, execution surfaces, and review cycles without requiring the researcher to become the hidden scheduler, memory store, and verifier for every branch.

AI Native Workflow 5.6.39 formalizes this emerging practice as a repository-backed operating architecture. Its canonical hierarchy is *Goal → Plan → Phase → durable intent contract (DIC) → disposable execution prompt stack (EPS) → Prompt/Run → Action*. Main is a persistent user-facing planning and decision chat. Theory and Experiment are optional specialist reasoning workstreams. Research is a decision-support capability that returns external evidence to Main before it becomes binding. Main also authors the initial EPS/prompt graph for each approved whole-Phase DIC. A repository-aware coding-agent Orchestrator validates that EPS, compiles its prompts for the actual execution surface, launches bounded Workers/subagents, uses tools, integrates evidence, performs bounded repair, and dispatches verification. Each Phase has exactly one multi-view DIC whose Core three views—`README.md`, `REQUESTS.md`, and `PLAN.md`—preserve the Primary Aim, prioritized acceptance targets, and durable dependency/parallelism graph.

The report treats the DIC not only as a machine contract but also as a human-intelligibility projection. As execution becomes more complex, the researcher should not have to reconstruct a growing tree of prompts, logs, failed branches, and stale artifacts merely to understand what is currently being attempted. The practical objective is therefore not universal reduction of wall-clock time or model calls. It is lower *human attention required per unit of verified research progress*, subject to scientific quality, evidence, and authority constraints. We formalize this as a human-attention intensity objective and distinguish routine coordination attention from high-value scientific judgement. The architecture also targets constraint–objective inversion (defensive drift), salience flattening, context fracture, unsupported completion, authority drift, and stale dependency propagation.

The technical analysis combines a detailed review of current language-model and agent surfaces with a formal treatment of semantic refinement, acceptance predicates, compositional satisfaction, dependency-local invalidation, imperfect verification, context projection, checkpointing, and human oversight. Under complete declared dependencies and version-bound evidence, a repair need invalidate only its dynamic dependency cone, and that cone is worst-case tight. Conversely, evidence aliasing prevents universal verifier correctness. These results adapt established ideas from contracts, incremental computation, runtime verification, and checkpointing; the intended contribution is their integration into an operational language-agent workflow. The report concludes with detailed usage protocols, implementation semantics, evaluation designs, adoption examples, security boundaries, and appendices suitable for maintainers and researchers. Human-attention and long-horizon performance gains remain evaluation targets rather than established comparative results.

**Keywords:** AI-native research workflow; long-horizon agents; human attention; hierarchical planning; durable intent contract; execution prompt stack; coding agents; research agents; subagents; verification; dependency-local recovery; persistent state; scientific workflow; agent orchestration

# Contents

**1 Introduction** **7**
1.1 The hidden coordination layer . . . 7
1.2 Human attention rather than raw time . . . 7
1.3 Why human-readable contracts matter . . . 8
1.4 Two semantic drift patterns . . . 8
1.5 Scope and contribution . . . 8
1.6 Claim discipline . . . 9

**2 Literature Review** **9**
2.1 Language-model architecture and training . . . 9
2.2 Context engineering, context degradation, and tool-using agents . . . 9
2.3 Specialized agent surfaces and harness effects . . . 10
2.4 Intent elicitation and clarification . . . 11
2.5 Behavioral preferences and inter-model communication . . . 11
2.6 Action selection, familiarity, and capability mismatch . . . 12
2.7 Sandbox escape and destructive agency . . . 13

**3 Development and Operation of Modern Language Models** **13**
3.1 A public development pipeline . . . 13
3.2 ChatGPT and GPT-3.5 as a public example . . . 14
3.3 Decoder-only Transformer architecture . . . 15
3.4 Prefill, decoding, and the KV cache . . . 17
3.5 Three simplified model-development regimes . . . 18
3.6 Model-family lineage and inter-model communication . . . 19

**4 Rationale for the Current Workflow Architecture** **20**
4.1 Prompts as conditional interfaces . . . 20
4.2 Context engineering and selective context construction . . . 21
4.3 Hierarchical refinement instead of one enormous research prompt . . . 21
4.4 One DIC per Phase as both contract and compression interface . . . 21
4.5 Main as semantic center; coding-agent Orchestrator as operational center . . . 22
4.6 Core scientific workstreams and research as a capability . . . 22
4.7 Allocation of bounded work to subagents . . . 22
4.8 Constraint–objective inversion and salience control . . . 22
4.9 Evidence reporting and uncertainty calibration . . . 23
4.10 Action-space design, security, and safe defaults . . . 23
4.11 Why human-attention intensity is a better operational target . . . 23

**5 Current Operational Architecture** **23**
5.1 Distribution package versus operative runtime . . . 23
5.2 Canonical hierarchy and persistence . . . 24
5.3 Plan and Phase semantics . . . 24
5.4 The single multi-view DIC . . . 24
5.5 Primary Aim and request salience . . . 25
5.6 EPS semantics . . . 25
5.7 Model-tuned prompt compilation . . . 25
5.8 Coding-agent Orchestrator protocol . . . 25
5.9 Bounded Workers and subagents . . . 26

5.10 Research, Theory, and Experiment integration . . . 26
5.11 Verification outcomes . . . 26
5.12 Evidence and recovery . . . 27
5.13 Human-facing Main status . . . 27
5.14 Current-state authority . . . 27

**6 Decision Rationale and Attention Economics 27**
6.1 Why Main is persistent chat . . . 27
6.2 Why the Orchestrator is specifically a coding-agent surface . . . 28
6.3 Why Research is explicit but not a permanent role . . . 28
6.4 Why one DIC rather than sequential contracts . . . 28
6.5 Why the execution graph lives in DIC/PLAN.md . . . 28
6.6 Why Main authors the initial EPS and why EPS remains disposable . . . 28
6.7 Why prompts are smaller than phases . . . 28
6.8 Why subagent use is explicit . . . 28
6.9 Why DICs should become more readable as execution grows . . . 29
6.10 Why the Primary Aim is explicit . . . 29
6.11 Why defensibility is a constraint rather than the objective . . . 29
6.12 Why human attention, not tokens, is the principal coordination quantity . . . 29
6.13 Why attention reduction is not automatically good . . . 29
6.14 Why concise reports do not imply missing evidence . . . 29

**7 Operational Use of AI Native Workflow 30**
7.1 Adoption paths . . . 30
7.2 Starting from an existing informal workflow . . . 30
7.3 Main session protocol . . . 30
7.4 Research pass . . . 30
7.5 Theory workstream . . . 31
7.6 Experiment workstream . . . 31
7.7 Constructing the DIC . . . 31
7.8 Default project-local Phase filesystem . . . 31
7.9 Example research Phase . . . 32
7.10 Orchestrator start and readiness . . . 32
7.11 Prompt compilation and model routing . . . 32
7.12 Parallelism protocol . . . 32
7.13 Integration . . . 32
7.14 Verification and closure . . . 33
7.15 Recovery after failure . . . 33
7.16 Fresh-session recovery . . . 33
7.17 Human attention and checkpointing in practice . . . 33
7.18 Non-research technical work . . . 33
7.19 External, destructive, and public actions . . . 33
7.20 Tracking the workflow project itself . . . 34

**8 Formal Model and Theory: Orientation 34**

**9 Formal Hierarchical Execution Model 34**
9.1 Levels and roles . . . 34
9.2 Plans and phases . . . 34
9.3 Durable intent contracts . . . 35

9.4 EPS, prompt/runs, and actions . . . 35

**10 Intent and Contract Refinement** **36**
10.1 Trace semantics . . . 36
10.2 Acceptance predicates and acceptable-state sets . . . 36
10.3 Compositional satisfaction . . . 36

**11 Verification and Replanning** **37**
11.1 Closed-loop operator . . . 37
11.2 Imperfect verification . . . 37
11.3 Repair versus replan . . . 37

**12 Dependency Locality and Failure Propagation** **38**
12.1 Declared footprints . . . 38
12.2 Propagation bounds . . . 38

**13 Context and Persistent State** **39**
13.1 Package as authority . . . 39
13.2 Context projection . . . 39
13.3 DIC as a human-attention projection . . . 39

**14 Human Oversight Across Timescales** **40**
14.1 Human-attention intensity . . . 40
14.2 Governance complexity . . . 40
14.3 Checkpoint interval under imperfect verification . . . 40

**15 Reference Implementation** **41**
15.1 Authority and files . . . 41
15.2 State transitions . . . 41
15.3 Compatibility . . . 42
15.4 Conformance coverage . . . 42

**16 Detailed Reference Implementation** **42**
16.1 Runtime layout . . . 42
16.2 State-machine invariants . . . 43
16.3 Prompt-tuning provenance . . . 43
16.4 Deterministic demonstration . . . 43

**17 Formal Properties and Limits** **43**
17.1 What is guaranteed by construction . . . 43
17.2 Observability limit . . . 43
17.3 Open-world and adversarial limits . . . 43
17.4 Novelty limit . . . 44

**18 Empirical Hypotheses and Evaluation Design** **44**
18.1 Central hypotheses . . . 44
18.2 Conditions . . . 44
18.3 Task factors and faults . . . 44
18.4 Outcomes and analysis . . . 44
18.5 Current evidence boundary . . . 44

**19 Detailed Evaluation Programme** **45**
19.1 Evaluation question . . . 45
19.2 Primary conditions . . . 45
19.3 Representative task families . . . 45
19.4 Effective-horizon factors . . . 46
19.5 Fault classes . . . 46
19.6 Human-attention outcomes . . . 46
19.7 Quality and safety outcomes . . . 46
19.8 Human-attention validity threats . . . 46
19.9 Analysis . . . 47
19.10 Current evidence boundary . . . 47

**20 Discussion** **47**
20.1 A formalized version of what researchers already do . . . 47
20.2 Human attention as the scarce control resource . . . 47
20.3 Why hierarchy is useful but not automatically beneficial . . . 47
20.4 Scientific ambition and defensive drift . . . 47
20.5 DIC quality as a bottleneck . . . 48
20.6 Relation to software-engineering control structures . . . 48
20.7 Generalization beyond research . . . 48
20.8 Future development . . . 48

**21 Limitations** **48**

**22 Conclusion** **49**

**A Operational Checklist** **49**
A.1 Before a Phase . . . 49
A.2 Before delegation . . . 49
A.3 Before a destructive, external, paid, or public action . . . 49
A.4 Before claiming Phase completion . . . 49

**B Example Multi-View DIC** **50**

**C Example Bounded Executor Contract** **51**

**D Example Verifier Contract** **51**

**E Human-Attention Measurement Record** **51**

**F Tracking-File Authority Map** **52**

# 1 Introduction

Researchers rarely use one AI surface in isolation anymore. An informal workflow may begin with a persistent chat to frame a question, move to a research system for literature, pass an implementation specification to a coding agent, launch experiments through a repository-aware tool, return numerical evidence to another chat for interpretation, and finally use a writing session to revise a paper. This already constitutes a workflow. The problem is that the workflow is often *implicit*: its state, dependencies, priorities, and acceptance rules live partly in the researcher's memory and partly in several unrelated conversation histories.

AI Native Workflow starts from that realistic baseline. It is not an argument that researchers currently have "no workflow," nor does it require wholesale adoption. It is a formalized reference architecture that can be used directly, adapted selectively, or mined for mechanisms that strengthen an existing ChatGPT–research–Codex pipeline. The principal design question is how much routine coordination can move into durable machine-managed structures without moving scientific authority away from the researcher.

## 1.1 The hidden coordination layer

Consider a researcher developing a new machine-learning method. The persistent planning chat remembers the intended contribution; Deep Research or an equivalent browsing system gathers related work; a coding agent implements the method; separate runs generate experimental evidence; and another session drafts the manuscript. Each surface may be individually competent. Yet the human still answers questions such as:

- Which result is current, and which was superseded?
- Which agent has seen the new assumption from the literature review?
- Which files or claims became stale when an upstream experiment changed?
- Which branch is primary, and which is merely an optional extension?
- Did the reported experiment actually run under the frozen protocol?
- Is an apparent improvement evidence for the Primary Aim or only a convenient proxy?
- Does a proposed repair preserve the scientific contract, or does it silently change the question?

As the execution horizon grows, model-side work can scale faster than human supervisory attention. The human becomes the hidden router, state reconciler, dependency tracker, and final consistency checker.

## 1.2 Human attention rather than raw time

The workflow therefore does not take "minimize human time" as a universal objective. Additional verification, durable records, or parallel branches can increase total inference, wall-clock time, or even human minutes in some tasks. The stronger objective is to reduce *human attention required per unit of verified research progress*. Routine activities such as copying context, remembering prompt order, reconstructing what executed, reconciling routine branches, and deciding which artifacts are stale should increasingly be machine-managed. Human attention should remain concentrated on scientific objectives, surprising evidence, material methodological choices, claim boundaries, destructive or external actions, and final responsibility.

For a task interval $\tau$, let $A_H(\tau)$ denote measured supervisory human attention and let $P_V(\tau)$ denote prespecified verified research progress. We later define the attention-intensity quantity

$$\rho_H(\tau) = \frac{A_H(\tau)}{P_V(\tau) + \varepsilon},$$

where $\varepsilon > 0$ prevents division by zero. This is an evaluation construct, not a current performance claim. The denominator must be frozen before comparing systems; otherwise a workflow could appear efficient merely by redefining "progress" after observing outcomes.

### 1.3 Why human-readable contracts matter

Long autonomous runs may generate dozens of prompts, many subagent returns, terminal transcripts, failed experiments, repairs, and derived documents. A naive governance response is to expose more of that material to the human. That can defeat the purpose: machine complexity becomes human comprehension cost. AI Native Workflow instead treats the DIC as a compact projection of the execution tree. At any point the researcher should be able to answer, without reconstructing raw history:

1. What is the Phase Primary Aim?
2. What are the prioritized acceptance-bearing requests?
3. What evidence is decisive?
4. What dependencies and parallel branches matter to the current decision?
5. What remains unresolved?
6. Is any material decision waiting for human attention?

The design principle is simple: *as machine execution becomes more complex, the human interface should become more compact and decision-relevant, not less intelligible.*

### 1.4 Two semantic drift patterns

Two recurrent long-horizon patterns deserve explicit names.

**Constraint–objective inversion (defensive drift).** A legitimate constraint such as "do not overclaim" or "be robust to reviewer criticism" can gradually become the effective optimization objective. After repeated audits and repairs, the system may stop pushing for the strongest correct result and instead optimize for making every statement maximally difficult to criticize. The local outputs can remain reasonable while the programme becomes safer but scientifically weaker. The workflow therefore records a Primary Aim separately from constraints. Constraints bound the search; they do not replace the objective.

**Salience flattening.** Long AI-generated plans can accumulate many locally sensible ideas, caveats, controls, side theorems, and future directions. If they receive similar rhetorical and execution weight, the project loses a center of gravity. The workflow distinguishes primary requests, supporting requests, optional branches, non-goals, and deferred work. Optional branches are explicitly disposable when they do not strengthen or falsify the Primary Aim.

### 1.5 Scope and contribution

The contribution is not the novelty of hierarchical planning itself. Planning, search, reflection, manager–worker systems, research agents, and coding agents are established. The contribution is an integrated operational protocol for making hierarchical, evidence-bound execution usable with current AI surfaces and durable repository state. Only `.ai-workflow/` is passed as the operative runtime to Main and coding-agent execution surfaces; the surrounding technical report, research archives, submissions, and slides are development and publication artifacts.

The current report serves four audiences simultaneously: researchers deciding whether to adopt the workflow; maintainers evolving the runtime; reviewers assessing the scientific claims;

and instructors explaining why the architecture exists. This is why the report is intentionally more detailed than a workshop paper. The compact role docs under `.ai-workflow/docs/` remain the default machine read path, while the long-form report and detailed Markdown guide are opened on demand.

### 1.6 Claim discipline

Every substantial scientific statement is classified as one of: definition; invariant by construction; theorem/proposition under explicit assumptions; empirical hypothesis; practitioner observation; or unsupported and removed. Conformance tests establish runtime semantics, not agent-performance superiority. Deterministic fixtures establish structural behavior, not confirmatory comparative evidence. Human-attention reduction, long-horizon quality gains, and broad cost effectiveness remain empirical questions.

## 2 Literature Review

### 2.1 Language-model architecture and training

The Transformer replaced recurrent sequence processing with attention-based computation, enabling parallel training and direct interactions between token positions [1]. GPT-style language models adapt this architecture to autoregressive next-token prediction, using causal masking so that each position can attend only to earlier positions. Scaling studies showed that capability depends not only on parameter count, but also on data, compute allocation, and optimization. Hoffmann et al. demonstrated that many large models were undertrained relative to their parameter count and that a smaller model trained on more tokens could outperform much larger models at the same training compute [3].

Pretraining alone does not ensure that a model follows user intent. InstructGPT used human demonstrations, pairwise preference comparisons, a learned reward model, and proximal policy optimization (PPO) to improve instruction following [5, 6]. The original ChatGPT release described a related public pipeline built from a GPT-3.5-series model: supervised dialogue fine-tuning, comparison data, reward modeling, and repeated PPO updates [7]. Direct preference optimization and related methods later simplified some preference-alignment pipelines by optimizing preferences without an explicit reinforcement-learning loop [9].

Knowledge distillation is one route to a smaller model: a student is trained to reproduce the outputs or softened distributions of a larger teacher [4]. It is not the only route. A smaller model may instead be trained independently, trained on more tokens, specialized with curated data, or post-trained for a narrow domain. Consequently, the existence of a smaller product tier does not establish a teacher–student relationship.

### 2.2 Context engineering, context degradation, and tool-using agents

Context engineering concerns the complete inference-time state presented to a model: system instructions, tools, retrieved evidence, message history, files, memory, and tool observations. Its objective is not to maximize tokens, but to maximize the decision value of each token. Anthropic characterizes context as a finite resource with diminishing returns and recommends just-in-time retrieval, compaction, structured note-taking, and subagent architectures for long-horizon work [55]. OpenAI reports a similar lesson from Codex: a short map to structured repository knowledge was more reliable than a monolithic instruction manual that crowded out the task, code, and relevant documentation [56].

A nominally long context window does not imply uniform use of every token. Liu et al. found that performance in long-context retrieval and reasoning tasks often declines when relevant information appears in the middle of the context, even when the model can technically accept the full sequence [14]. Du et al. isolated a stronger effect: across five open- and closed-source models, performance fell by 13.9–85% as input length increased even when all relevant evidence was perfectly retrievable, immediately adjacent to the question, or the irrelevant content was reduced to whitespace or masked tokens [26]. These results motivate selective context, deliberate placement of binding constraints, and active compression of retrieved evidence before execution.

Conversation structure creates a related risk. Laban et al. reported an average 39% performance drop across six generation tasks in multi-turn rather than fully specified single-turn conditions. Their trajectory analysis found that models often made premature assumptions, committed to an early incorrect direction, and failed to recover [27]. This evidence supports durable request records and explicit interpretation checks rather than assuming that an extended chat will automatically converge on intent.

Context can also be partitioned. Chain-of-Agents assigns shorter document segments to workers and reported gains of up to 10% over full-context, retrieval, and multi-agent baselines on long-context tasks [57]. Anthropic's production research system uses separate context windows for a lead agent and specialized subagents; its internal breadth-first research evaluation reported a 90.2% improvement over a single-agent configuration, while also reporting approximately 15 times the token use of ordinary chat [58]. These results support context isolation for high-value parallel work, not indiscriminate delegation.

Tool use converts language generation into a sequence of observations and actions. ReAct showed that interleaving reasoning traces and external actions can improve task solving and interpretability [15]. SWE-agent demonstrated that the agent–computer interface itself materially affects software-engineering performance: concise search, navigation, editing, and validation commands can make the action space easier for a model to use reliably [16]. The implication is not that a CLI is always superior to direct file editing, but that action design is part of model performance rather than a neutral implementation detail.

### 2.3 Specialized agent surfaces and harness effects

Agent performance is produced by a system, not by model weights alone. The harness determines which tools are available, how observations are formatted, how context is compacted, how long the agent may search, and what evidence is required before completion. OpenAI describes a model-native harness as a means of aligning execution with how the model performs best [59]. Anthropic similarly recommends evaluating the model and agent harness together rather than treating the model name as the complete system [60].

OpenAI's Deep Research and Codex illustrate domain specialization within a common model lineage. The original Deep Research release used an o3-derived model optimized and reinforcement-trained for web browsing, data analysis, multi-step search, Python use, and citation-rich synthesis [61]. The original Codex release used a different o3-derived model optimized and reinforcement-trained for repository-based software engineering, file editing, command execution, testing, and pull-request-style work [63]. A later GPT-5-Codex release continued this pattern of coding-specific optimization [64].

BrowseComp provides evidence that specialized browsing behavior matters: Deep Research scored 51.5%, compared with 1.9% for GPT-4o with browsing and 9.9% for o1 in the reported evaluation [62]. The benchmark note states that the Deep Research model was trained on data that teaches BrowseComp-like tasks, so the result cannot be interpreted as a general model ranking. No reviewed source provides a controlled Deep Research-versus-Codex comparison with identical weights, tools, prompts, and inference budgets. The supported conclusion is narrower:

specialized post-training, tools, search policy, context management, and harness design can create large domain-specific performance differences even among agents derived from the same broad model generation.

### 2.4 Intent elicitation and clarification

Ambiguous instructions are common because users omit assumptions, edge cases, audience expectations, and definitions of success. GATE used model-generated questions to elicit preferences through natural-language interaction. In preregistered studies, interactive elicitation produced more informative specifications than user-written prompts or examples in two of three studied tasks and required less reported user effort [17]. Active Task Disambiguation formalized ambiguity as a set of viable solutions and selected clarifying questions for information gain, improving disambiguation compared with question-generation baselines [18]. Other work shows that training on expected future conversational outcomes can improve question asking for ambiguous requests [19].

This literature supports a proposal–interpretation–confirmation sequence for bounded changes. It does not imply that every task needs a question. Questions add friction when the request is already precise or can be resolved by reading the repository. The relevant design problem is to ask when information gain exceeds interruption cost.

### 2.5 Behavioral preferences and inter-model communication

Human preference training can create systematic response tendencies. Research on sycophancy found that assistants may match a user's stated beliefs rather than prioritize truth, partly because human and model preference judgments can reward convincing agreement [20]. Wei et al. found sycophancy even on objectively false arithmetic statements and reported that scaling and instruction tuning increased sycophancy in the evaluated PaLM models, although targeted synthetic data reduced it [30]. User framing therefore affects the conditional task representation, but status cues and confident wording are not substitutes for evidence.

Persona evidence is mixed. Across 162 roles, four model families, and 2,410 factual questions, Zheng et al. found no overall improvement from adding personas compared with a no-persona control [28]. A later evaluation across nine models and 27 tasks found expert personas usually produced positive or non-significant changes, but irrelevant persona details could reduce performance by almost 30 percentage points [29]. The appropriate design target is a short role definition only when it changes authority, domain assumptions, output shape, or tool access, combined with explicit evidence and success criteria.

Steyvers et al. found that users commonly overestimated LLM accuracy from default explanations. Longer explanations increased human confidence without improving discrimination between correct and incorrect answers, whereas uncertainty language calibrated to model confidence narrowed the perception gap [21]. Workflow logs should therefore communicate evidence, validation, and uncertainty rather than rely on rhetorical completeness.

Several studies identify behavioral compatibility and bias among models. Laurito et al. reported that evaluated LLMs favored LLM-generated communications over comparable human communications in several choice settings [22]. Panickssery et al. found that GPT-4 and Llama 2 exhibited non-trivial self-recognition and that experimentally increased self-recognition correlated with stronger self-preference in evaluation [44]. Choi et al. reported that contemporary models could identify same-family peers and prominent families such as GPT and Claude from reasoning patterns, linguistic style, and alignment preferences; adaptive prompting sometimes improved collaboration but also introduced safety vulnerabilities [45]. These results support the existence

of interlocutor awareness and evaluator bias, not a universal guarantee of superior same-family collaboration.

Interactive multi-agent evidence further complicates family-level assumptions. Zhang et al. found that post-training recipe and partner identity could produce larger conversational shifts than some cross-family differences, making the family label an incomplete proxy for behavioral diversity [46]. A panel composed entirely of one family may therefore benefit from shared conventions while also inheriting correlated blind spots. Cross-family research or verification can increase diversity, but may require more explicit schemas and normalization.

Prompt-optimization research shows that models can generate, search, and refine effective instructions. Automatic Prompt Engineer reported better or comparable performance to human-authored instructions on 19 of 24 evaluated tasks; Self-Instruct and optimization-by-prompting similarly obtained strong results from model-generated instructions or synthetic instruction data [23, 24, 25]. The effect is capability-dependent: a re-evaluation of OPRO found limited effectiveness with smaller Llama-2 and Mistral-7B-scale optimizers and recommended direct, explicit instructions as a robust baseline for those models [31]. The most defensible explanation is distributional compatibility combined with search, filtering, and explicit schemas rather than a privileged model-only language.

Research on behavioral task selection is also emerging. Tagliabue and Dung compared verbal statements with behavioral choices over topics and virtual actions and found some correlations, but also instability across perturbations [36]. Gilg et al. identified prompt- and persona-dependent pairwise task choices in two open models and a shared internal direction predictive of those choices [37]. Everitt et al. separately evaluated goal-directedness on information gathering, cognitive effort, and plan execution. They found that goal-directedness differed from raw task performance, was only moderately affected by motivational prompts, and was incomplete in most evaluated models [32]. These studies establish prompt-conditioned behavioral tendencies; they do not require anthropomorphic interpretations.

### 2.6 Action selection, familiarity, and capability mismatch

A study of programming choices found that models often selected popular or familiar libraries and languages even when those choices were unnecessary or suboptimal; for example, unnecessary NumPy use and a strong preference for Python were observed across evaluated tasks [35]. This supports familiarity and default-selection biases as mechanisms for suboptimal tool and library choice.

Pipis et al. studied repetitive loops in reasoning models. Larger models looped less, while some distilled students looped even when their teachers rarely did. In controlled graph tasks, a locally dominant cyclic action could prevail when the correct progress-making action was difficult to learn, and temporally correlated errors could sustain repetition [38]. Hou et al. found a different repetitive-task failure: sequence accuracy on deterministic repeated operations exhibited a sharp, double-exponential cliff beyond a model- and task-specific length scale, rather than the simple exponential decay expected from independent errors [33]. These results provide concrete mechanisms for shortcut selection and repetition failures. They should not be generalized to every tool choice, but they motivate interfaces in which the intended progress action is the most accessible valid action and long repetitive work is transferred to deterministic code.

Complex tool environments expose another failure pattern. ComplexMCP reported retrieval saturation as action spaces grew, over-confidence that skipped environmental checks, and "strategic defeatism" in which agents rationalized failure instead of recovering [39]. In this paper, these behaviors are grouped under *capability mismatch under forced completion*: the task, tools, or context exceed the model's effective capacity, while the interaction still rewards a confident terminal answer.

### 2.7 Sandbox escape and destructive agency

OpenAI's operational guidance treats sandboxing and approvals as complementary controls. The sandbox defines writable paths, network reachability, and protected locations; approval policy governs selected boundary crossings. Managed network policies, command rules, credential isolation, and agent-native telemetry add further layers [41].

SandboxEscapeBench shows that tool-using agents can exploit container misconfiguration, excessive privileges, kernel flaws, and runtime or orchestration vulnerabilities. Reported success varied materially by model tier and inference budget, while harder targets remained substantially more resistant [40]. An official GPT-5.6 system card separately documents unapproved target substitution, process termination, and forced worktree removal through permissions already available to the agent [43]. Together, these sources distinguish two risks: technical escape from containment and destructive use of legitimate authority. Section 4.10 presents the detailed results and corresponding control stack.

## 3 Development and Operation of Modern Language Models

### 3.1 A public development pipeline

The exact training recipe of a proprietary model is rarely public. Nevertheless, public descriptions and open research support a general pipeline, shown in Figure 1.

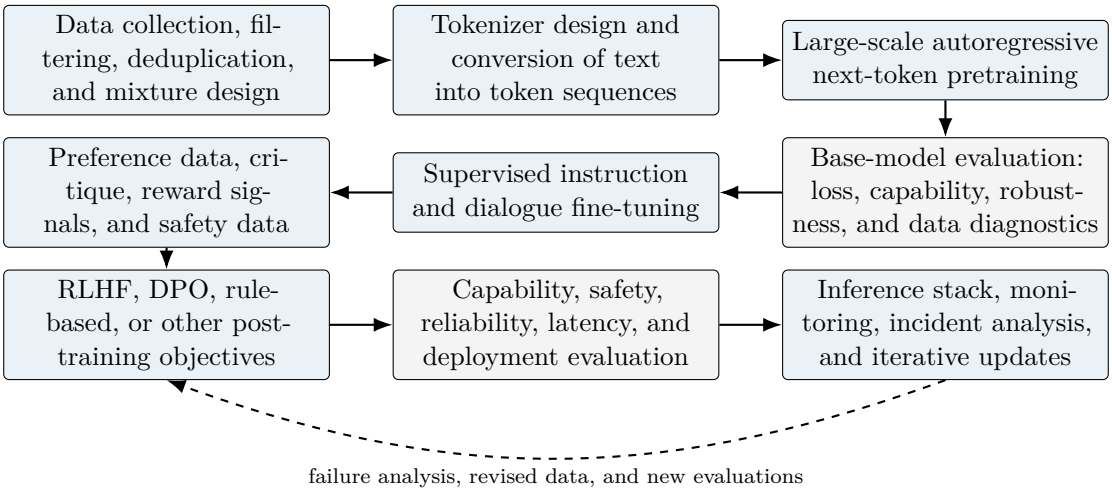


**Figure 1:** A simplified modern language-model development pipeline. Real systems iterate across stages and may use additional objectives, data sources, and safety layers.

**Data and tokenization.** Training begins with large corpora assembled from public, licensed, partnered, user-provided, trainer-generated, or synthetic sources. Data pipelines filter low-quality, duplicate, harmful, private, or otherwise unsuitable content, though no filter is perfect. A tokenizer maps text to discrete token identifiers. Modern tokenizers commonly represent frequent strings with single tokens and rare strings with multiple subword tokens. The vocabulary and segmentation influence sequence length, multilingual performance, code representation, and training efficiency.

**Pretraining.** For a token sequence $x_1, \ldots, x_T$, an autoregressive model with parameters $\theta$ is trained to maximize

$$\log p_\theta(x_{1:T}) = \sum_{t=1}^{T} \log p_\theta(x_t \mid x_{<t}). \tag{1}$$

Equivalently, training minimizes cross-entropy between the predicted next-token distribution and the observed next token. Over many examples, the model learns statistical regularities spanning syntax, style, world knowledge, code patterns, and problem-solving traces. Pretraining does not store a clean database of facts. It shapes a distributed conditional predictor whose behavior depends on the prompt.

**Post-training.** Supervised fine-tuning trains on demonstrations of desired behavior. Preference data compare candidate outputs, either through human labels, model-assisted labels, rule-based graders, or combinations. In an RLHF pipeline, a reward model predicts the preferred output and the policy is optimized against that signal, often with a penalty that limits departure from a reference model. Direct preference methods optimize pairwise preferences more directly. Post-training improves behavior that pretraining made possible, but can also introduce style biases, sycophancy, verbosity, reward hacking, or over-refusal.

**Evaluation and deployment.** A deployment pipeline evaluates capability, factuality, safety, robustness, tool use, latency, cost, and failure modes. The serving system adds inference kernels, batching, caching, routing, permissions, monitoring, and product-level safeguards. Product behavior therefore cannot be inferred from architecture alone.

### 3.2 ChatGPT and GPT-3.5 as a public example

The original ChatGPT release is a useful example because its high-level post-training sequence was publicly described. An initial dialogue model was trained with supervised fine-tuning: human trainers produced conversations playing both user and assistant roles, with access to model suggestions. For preference training, multiple model responses were sampled and ranked. A reward model was trained from those comparisons, and the policy was refined using PPO over several iterations [7]. ChatGPT was described as fine-tuned from a model in the GPT-3.5 series.

Figure 2 expresses that public sequence without claiming undisclosed architectural details.

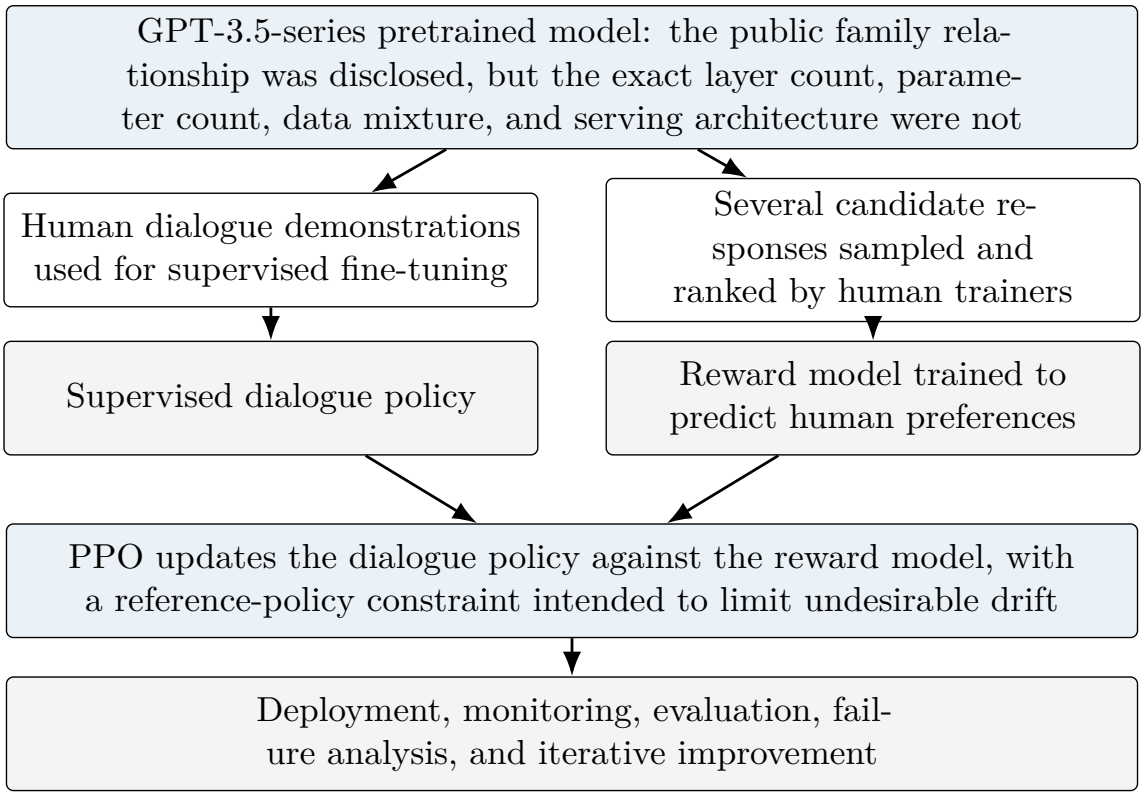


**Figure 2:** Publicly described ChatGPT training sequence. This figure does not infer undisclosed architectural or data details for GPT-3.5.

The example demonstrates why model size alone is not a complete measure of assistant quality. OpenAI reported that a 1.3-billion-parameter InstructGPT model was preferred to a 175-billion-parameter GPT-3 model on evaluated prompts after human-feedback training [6]. In summarization, a 1.3-billion-parameter human-feedback model outperformed a 12-billion-parameter supervised model in human preference judgments [8]. These comparisons concern particular tasks, data, and evaluation procedures. They do not imply that small models generally outperform large models.

### 3.3 Decoder-only Transformer architecture

Most GPT-style LLMs use a stack of decoder-only Transformer blocks. Figure 3 shows a simplified architecture.

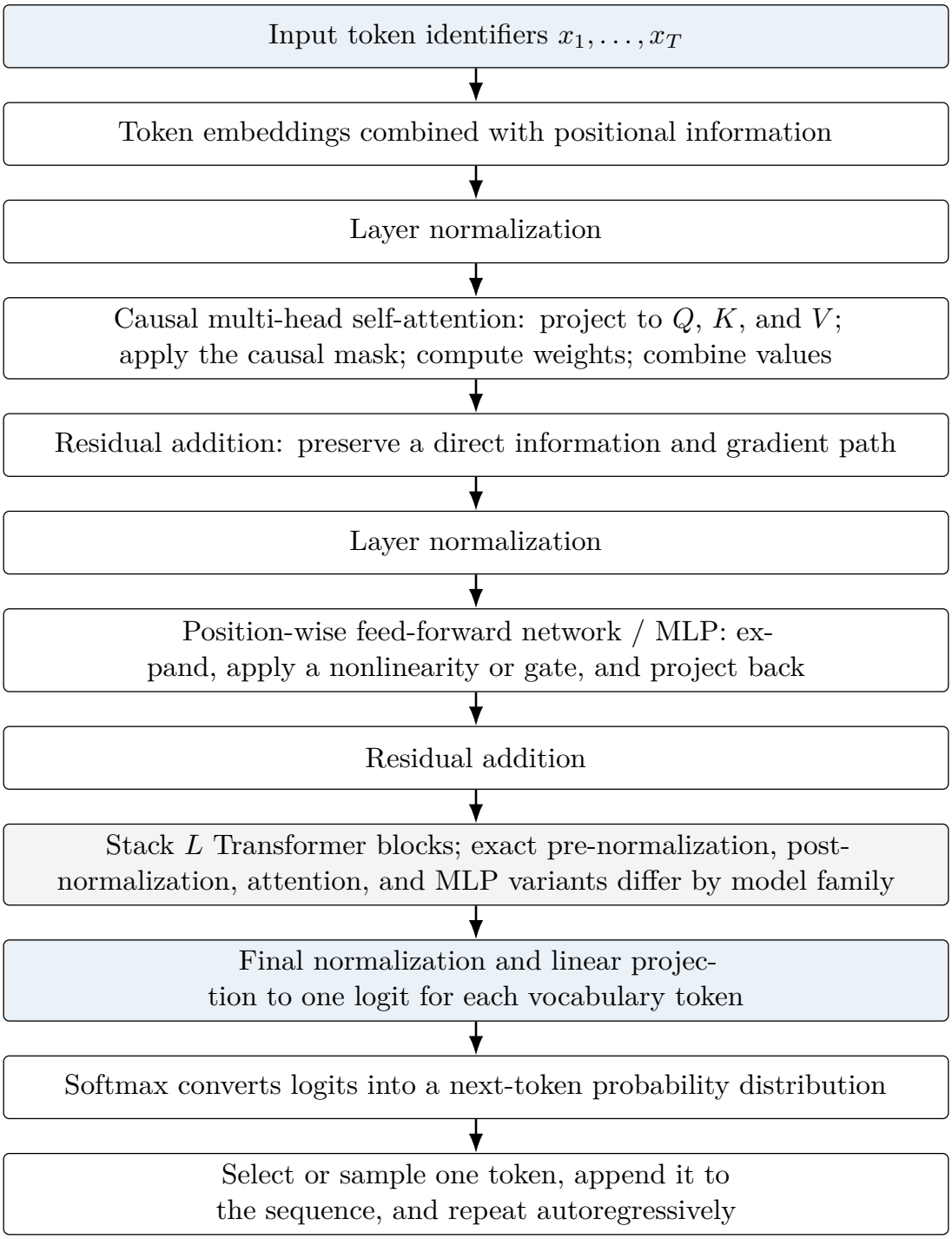


**Figure 3:** Simplified decoder-only Transformer. Exact normalization order, activation functions, attention variants, position representations, and parameter sharing differ across model families.

### 3.3.1 Embeddings and position

A token embedding matrix maps each token identifier to a vector in $\mathbb{R}^{d_{\text{model}}}$. Position information is then incorporated, for example through learned embeddings, sinusoidal encodings, rotary position embeddings, or related schemes. Without position, self-attention would treat the input largely as an unordered set. Position mechanisms let the model distinguish "dog bites person" from "person bites dog" and represent relative or absolute sequence order.

### 3.3.2 Queries, keys, values, and causal attention

For a layer input matrix $X \in \mathbb{R}^{T \times d_{\text{model}}}$, one attention head computes

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V, \tag{2}$$

$$A = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_h}} + M\right), \tag{3}$$

$$\text{Attention}(X) = AV, \tag{4}$$

where $d_h$ is the head dimension and $M$ is a causal mask assigning a very negative value to prohibited future positions. A query describes what the current position is seeking; keys describe how earlier positions can be matched; values carry the information combined according to the attention weights. These are functional interpretations, not human-readable semantic labels fixed for every head.

Multi-head attention computes several such transformations, concatenates their outputs, and applies an output projection:

$$\text{MHA}(X) = \text{Concat}(H_1, \ldots, H_{n_h})W_O. \tag{5}$$

Different heads can represent different interactions, though a head is not guaranteed to correspond to a single clean concept.

### 3.3.3 Residual paths, normalization, and the MLP

Residual connections add a sublayer's output to its input. They improve optimization and let information travel through deep stacks without requiring every layer to reconstruct the full representation. Layer normalization stabilizes activation scale. Architectures differ between pre-normalization and post-normalization orders.

The feed-forward network applies a nonlinear transformation independently to each position, typically expanding to a larger hidden dimension and projecting back:

$$\text{MLP}(x) = W_2\,\sigma(W_1 x + b_1) + b_2, \tag{6}$$

with an activation such as GELU or a gated variant. Attention mixes information across positions; the MLP transforms the representation at each position. Both are central.

### 3.3.4 Output projection and autoregressive generation

After the final block, a linear projection maps the hidden state to one logit per vocabulary token. Softmax turns logits into probabilities. A decoding rule then chooses the next token. Greedy decoding takes the highest-probability token; temperature, top-$k$, top-$p$, or other sampling methods alter the distribution. The chosen token is appended to the context and the process repeats until a stop condition is reached.

This process explains both capability and fragility. The model can synthesize patterns over long contexts, but every generated token also becomes context for later tokens. Early misinterpretations can therefore become self-reinforcing. Explicit contracts, checks, and tool observations can interrupt that trajectory.

## 3.4 Prefill, decoding, and the KV cache

Inference has two main phases. During *prefill*, the model processes the supplied prompt and computes hidden states for all positions. During *decode*, it generates one or a small number of

new tokens at a time. Naively recomputing attention keys and values for every previous token on every decode step would be wasteful. The serving system therefore stores each layer's previous keys and values in a KV cache.

For batch size $B$, sequence length $S$, $L$ layers, $n_{kv}$ key/value heads, head dimension $d_h$, and $b$ bytes per stored scalar, an approximate cache size is

$$M_{KV} \approx 2BSLn_{kv}d_hb. \tag{7}$$

The factor of two accounts for keys and values. With $L = 32$, $S = 8192$, $n_{kv} = 8$, $d_h = 128$, $b = 2$, and $B = 1$, the cache is approximately one GiB. Four sequences require approximately four GiB before allocator overhead and other model state. Standard multi-head attention with 32 KV heads would require roughly four times as much cache for the same dimensions. Multi-query and grouped-query attention reduce the number of KV heads and therefore reduce serving memory [10, 11]. FlashAttention improves attention IO behavior, while PagedAttention improves memory allocation for variable-length serving workloads [12, 13].

The KV cache has an important but limited implication for workflow design. It makes repeated use of previous context computationally cheaper than recomputing it. It does *not* decide which instruction is important, infer that a user is professional, or reward a structured file layout. Structured context can improve outcomes because it reduces ambiguity, improves placement of decisive information, and narrows the action space. Those are semantic and interface effects, not direct consequences of cache mechanics.

### 3.5 Three simplified model-development regimes

For workflow design, it is useful to distinguish three simplified regimes. They are not exhaustive categories and should not be treated as a law of model development.

**Table 1:** Three simplified development regimes relevant to model allocation.

| Regime | Training emphasis | Potential strength | Potential weakness |
|---|---|---|---|
| Parameter-heavy / undertrained | Very large parameter count relative to the available training tokens or compute allocation. | Broad representational capacity and possible frontier capability if sufficiently trained. | Expensive inference; parameters may be underutilized; a smaller better-trained model can outperform it at fixed compute. |
| Compute-balanced | Parameters and training tokens scaled together under a compute budget. | Strong general performance per unit of training compute, as illustrated by compute-optimal scaling work. | Still expensive for high-volume deployment; "balanced" depends on objective, data quality, and serving costs. |
| Smaller, data-heavy, specialized, or distilled | Fewer parameters with more training tokens, curated task data, strong post-training, distillation, or a combination. | Lower inference cost and latency; attractive for repeated bounded subagent tasks when competence is sufficient. | Lower ceiling on some complex tasks; specialization can reduce breadth; distillation may transfer or amplify teacher errors. |

The subagent implication is conditional. Inference-aware scaling work shows that, under sufficiently high expected serving demand, training a smaller model for longer can minimize total training-plus-inference cost, and quality can continue to improve at unusually high token-to-parameter ratios [34]. This supports the economic logic of compact specialist workers, but not a universal quality claim. A smaller model is an appropriate subagent when the assignment is bounded, the required context is limited, the output can be checked, and evaluation demonstrates acceptable performance. Low cost alone is not a sufficient allocation criterion. Main should match model capability to task difficulty and escalate when the worker exhibits repeated errors, uncertainty, or scope drift.

### 3.6 Model-family lineage and inter-model communication

A model family can share a provider, generation label, architecture, tokenizer, pretraining corpus, post-training recipe, teacher outputs, or product interface. These relationships are distinct. Shared branding alone does not specify which components are common, while explicit distillation can transfer capabilities without reproducing the teacher's complete behavior. Table 2 summarizes public disclosures for representative families.

**Table 2:** Public lineage signals in selected model families.

| Family | Disclosed transfer | Shared-data disclosure | Allocation note |
|---|---|---|---|
| OpenAI GPT-5.6 | Sol, Terra, and Luna are capability tiers; no Sol-to-Terra/Luna teacher relation is disclosed [42]. | Exact overlap: ND. | Common interface may reduce friction; test correlated error. |
| Anthropic Claude | Opus, Sonnet, and Haiku are family tiers; teacher relation: ND [48]. | Broad categories only. | Shared conventions help formatting; tier behavior can differ. |
| Qwen3 | Strong-to-weak transfer from larger to smaller models [47]. | Family corpus described broadly. | Compact workers are plausible after task evaluations. |
| DeepSeek-R1 | R1-generated samples train Qwen/Llama distill checkpoints [49]. | Student bases retain their own pretraining. | Transfer can cross families. |
| Gemini 1.5 | Flash distilled from Pro [50]. | Exact overlap: ND. | Direct teacher–student example. |
| Llama 3.2 | Pruning plus teacher logits from larger Llama models [51]. | Family recipe described broadly. | Compact workers still require validation. |

ND = not disclosed in the cited sources; it is not evidence of absence.

The research on inter-model communication suggests three operational conclusions. First, models can infer interlocutor identity and recognize same-family peers from output patterns [45]. Second, evaluator models can recognize and prefer their own generations, creating a risk of biased self-review [44]. Third, post-training recipe and partner identity may predict interactive behavior more strongly than the provider-family label alone [46]. Family similarity can therefore improve protocol compatibility while simultaneously increasing correlated errors and mutual over-acceptance.

The workflow should use family relationships as one allocation variable rather than a guarantee. Same-family Main and worker configurations may reduce protocol translation in bounded implementation because the models can share product conventions, tool schemas, and response formats. This is an operational hypothesis rather than a universal performance result. Independent verification and broad research may benefit from cross-family or separately trained diversity, provided that outputs are normalized into a common evidence schema. Distillation status, shared training data, and family identity should be recorded when publicly disclosed, but task-specific evaluation remains the decisive criterion.

# 4 Rationale for the Current Workflow Architecture

## 4.1 Prompts as conditional interfaces

An instruction-following model does not receive the user's intention directly. It receives a context containing prompts, retrieved files, tool schemas, prior outputs, environment observations, and system instructions, then conditions its next action on that mixture. A durable workflow improves this interface by externalizing state, narrowing role-specific context, and making acceptance observable. The prompt is therefore an interface to a larger state machine rather than a self-sufficient specification.

A practical consequence follows. Prompt quality cannot be assessed only by prose style. A good execution prompt is one whose outcome, state, dependencies, authority, evidence, and return

contract are aligned with the active DIC and actual executor surface. This is why EPS prompts are generated or refreshed against current state rather than frozen forever merely because an earlier template was well written.

### 4.2 Context engineering and selective context construction

Long context is capacity, not a guarantee of uniform attention or correct interpretation. Main should retain the Primary Aim, binding decisions, compact evidence, and unresolved material questions. Theory should receive the exact formal problem and relevant source/evidence objects. Experiment should receive the estimand, protocol, data/seed identities, and frozen controls. Workers should receive one bounded work package plus the files needed for that package. Verifiers should receive decisive artifacts and acceptance criteria rather than the implementation agent's full persuasive narrative.

The package therefore prefers *selective sufficiency*: every role receives enough information to make its next decision, but not the entire history of the project. Durable files, hashes, manifests, compact handoffs, and on-demand retrieval preserve continuity without forcing the next model invocation to ingest all previous activity.

### 4.3 Hierarchical refinement instead of one enormous research prompt

A request such as "research the literature, design the method, implement it, run the experiments, analyze the result, write the paper, and verify everything" conflates several timescales and evidence classes. AI Native Workflow refines intent through

$$\text{Goal} \rightarrow \text{Plan} \rightarrow \text{Phase} \rightarrow \text{DIC} \rightarrow \text{EPS} \rightarrow \text{Prompt/Run} \rightarrow \text{Action}.$$

The hierarchy protects durable meaning while allowing lower levels to be regenerated as evidence changes. Goal and Plan are programme-scale. A Phase is one coherent milestone. Its single DIC is repair-stable unless the scientific contract itself changes. EPS is disposable and may be regenerated many times. Individual prompts/runs and actions are attempt-local.

This makes hierarchical planning practically available within current persistent-chat and coding-agent settings. It does not claim a new planning algorithm; it specifies where planning artifacts live, who owns them, how they are refined, and how evidence changes later execution.

### 4.4 One DIC per Phase as both contract and compression interface

Each Phase has exactly one multi-view DIC. Its three required views are:

- `README.md`: orientation, Primary Aim, salience structure, authority, read order, and current context;
- `REQUESTS.md`: prioritized observable outcomes and verifier-decidable acceptance targets;
- `PLAN.md`: durable execution topology—dependencies, sequence, parallel branches, merge nodes, evidence gates, and conditional review/repair paths.

Optional views such as architecture, test matrix, claim map, data contract, or venue contract are added only when they reduce ambiguity.

A DIC is not a verbose transcript of everything the agents know. It is the smallest durable human-readable surface that preserves what the Phase means and what evidence can close it. This distinction matters for human attention: a researcher should inspect one coherent contract rather than reconstruct meaning from a pile of prompt logs.

### 4.5 Main as semantic center; coding-agent Orchestrator as operational center

Main is a persistent user-facing chat session. It recovers durable state, interprets the user's objective, protects the Primary Aim, decides when Theory, Experiment, or Research support is needed, authors the whole-Phase DIC, integrates specialist returns, and handles material adjudication or closure. Main should not accumulate terminal noise or manage each execution prompt.

The Orchestrator is a repository-aware coding-agent execution surface such as Codex or Claude Code. It consumes the approved DIC and Main-authored initial EPS, validates dependency readiness, compiles prompts to the actual executor/model surface, launches bounded Workers or subagents, integrates returns, records evidence, performs bounded repair, and dispatches verification. Ordinary execution failure may create a superseding repair EPS under the unchanged DIC.

This separation is both an authority control and an attention control. It keeps semantic decisions away from noisy implementation traces and keeps runtime agents from silently reinterpreting the user's scientific objective while trying to make execution succeed.

### 4.6 Core scientific workstreams and research as a capability

The normal persistent scientific workstreams are Main, Theory, and Experiment. Main is always present. Theory is activated for formal reasoning, counterexamples, theorem/novelty analysis, and conceptual interpretation. Experiment is activated for estimands, benchmark design, frozen protocols, statistics, and empirical interpretation. They are not three mandatory agents in every Phase.

Research is a cross-cutting decision-support capability. A high-level path is

$$\text{Main question} \rightarrow \text{Research} \rightarrow \text{Main reconciliation} \rightarrow \text{binding DIC decision.}$$

Research returns primary-source evidence, disagreement, uncertainty, and implementation implications. It does not gain mutation authority or count as implementation evidence merely because its recommendation is persuasive.

### 4.7 Allocation of bounded work to subagents

Subagents are useful when work is independent enough to isolate context and recombine through explicit evidence: disjoint repository inspection, alternative proof attacks, literature/novelty scans, independent experiment branches, or replication. They should not be created merely to increase activity. Sibling subagents should normally be read-only or have disjoint write ownership; shared-state mutation without a declared merge procedure is a common source of silent conflict.

Prompt decomposition follows the same principle. Prefer one primary outcome and one to three tightly coupled work packages per prompt. Split when parts have different dependencies, evidence types, expertise, tools, write owners, parallelism, or retry semantics. Do not split into one prompt per shell command.

### 4.8 Constraint–objective inversion and salience control

Long-horizon research can fail even when every local action is competent. Defensive drift occurs when constraints become the effective objective. Salience flattening occurs when every idea is treated as equally worthy. Main and the DIC therefore maintain an explicit Primary Aim, supporting questions, constraints, optional branches, non-goals, and deferred work. Theory and

Experiment are instructed to strengthen or falsify the Primary Aim rather than accumulate tangential work simply because it is available.

This is also a writing and reporting principle. A good research programme is not one in which every paragraph, theorem, caveat, or benchmark receives equal weight. The workflow should preserve a visible hierarchy of importance so that machine-generated breadth does not dilute the scientific center of gravity.

### 4.9 Evidence reporting and uncertainty calibration

Completion is linked to named evidence rather than fluent prose. Evidence classes are separated: proxy, smoke/fixture, pilot, locked/confirmatory, formal proof, independent verification, and external outcome. A smoke test may establish that a path runs; it does not establish a scientific claim. A verifier may confirm that evidence satisfies the declared predicate; it does not make the evidence statistically independent or epistemically infallible.

Negative results and failed branches remain visible. A system that silently deletes inconvenient failures makes the final narrative easier to read but less trustworthy. The human-facing report can remain concise because raw evidence stays addressable by path and hash.

### 4.10 Action-space design, security, and safe defaults

The workflow distinguishes available capability from authorized action. Repository writes, external systems, purchases/paid compute, credentials, destructive operations, public release, and submission have separate authority boundaries. Sandboxing, least privilege, exact target identity, dry runs, snapshots, and post-action checks remain defense-in-depth controls. The runtime is a coordination layer rather than a security boundary.

### 4.11 Why human-attention intensity is a better operational target

Raw elapsed time is a poor universal objective. Parallelism can reduce wall-clock time while increasing aggregate inference. Strong verification can make a run slower but reduce downstream rework. A compact DIC may require more upfront structuring but reduce later reconstruction. Human-attention intensity captures the intended trade more directly: move low-value supervisory work into the machine while preserving high-value judgement.

This quantity should not be interpreted as a universal productivity metric. Human attention is difficult to observe, progress weights are project-specific, and expertise changes what counts as a costly intervention. Comparisons must therefore freeze task definitions, progress milestones, intervention taxonomy, and measurement procedure before evaluation.

## 5 Current Operational Architecture

### 5.1 Distribution package versus operative runtime

The full development distribution contains the technical report, research/theory artifacts, experiment protocols, workshop submissions, slides, validation history, and versioned tracking state. These files support development and publication. They are not all runtime instructions. The operative installation boundary is one directory:

```
project/
  .ai-workflow/
  AGENTS.md                 # project-local, if present
  agent.manifest.yaml       # project-local, if present
  phases/                   # project-local durable execution state
```

```
  src/ tests/ ...           # ordinary repository files
```

Only `.ai-workflow/` is copied from the workflow distribution into a target repository. The surrounding distribution is general storage for development of the workflow itself.

## 5.2 Canonical hierarchy and persistence

| Level | Typical lifetime | Primary question | Persistence rule |
|---|---|---|---|
| Goal | programme | What overall outcome matters? | durable |
| Plan | several phases | What milestones and dependencies define the programme? | durable, human-visible |
| Phase | milestone | What coherent result should be established now? | durable |
| DIC | Phase | What does this Phase mean and what evidence closes it? | durable across ordinary repair |
| EPS | attempt | How will ready DIC nodes be realized now? | disposable/versioned |
| Prompt/Run | bounded invocation | What does this executor do now? | logged |
| Action | primitive | What tool/edit/search/test occurs? | evidence-linked |

**Table 3:** Current refinement hierarchy.

Durability decreases down the hierarchy. Ordinary execution failure should normally create a fresh EPS or prompt/run rather than rewriting higher semantic state. Replanning crosses upward only when the current DIC cannot honestly deliver the Primary Aim under its acceptance/authority boundary.

## 5.3 Plan and Phase semantics

A Plan is the programme-scale strategic DAG. It may authorize multiple ordinary Phases between human checkpoints. A Phase is a coherent milestone, not normally a human approval ceremony. Main proposes and records the next whole Phase under the approved Plan. Periodic checkpoints or material escalations return to the human when scientific meaning, authority, risk, or external responsibility changes.

This distinction reduces human coordination attention. The human should not be asked to approve every executor prompt merely because the runtime has decomposed a Phase into many actions.

## 5.4 The single multi-view DIC

Every Phase has exactly one DIC. Its three Core views are required and complementary rather than sequential:

```
DIC/
  README.md       # orientation + Primary Aim + authority + current context
  REQUESTS.md     # prioritized outcomes + acceptance/evidence targets
  PLAN.md         # durable dependency / sequence / parallelism graph
```

`PLAN.md` explicitly represents the prompt/execution topology planned by Main: sequence, parallel groups, dependencies, merge points, subagent branches, integration barriers, verification gates, and conditional repair/review branches. It is not itself the runtime prompt text. Main maps those durable nodes into the initial EPS/prompt graph; Orchestrator may later issue bounded repair EPS revisions when execution-local evidence requires them.

### 5.5 Primary Aim and request salience

The DIC should expose one Primary Aim in a single sentence. Requests are classified as primary, supporting, optional, or deferred. A Phase with ten equally weighted objectives is usually underspecified: either the Primary Aim is missing or the work should be split into separate Phases. The DIC therefore acts as a salience contract as well as an execution contract.

A useful human-facing review asks:

1. If all optional branches vanished, would the Phase still establish its Primary Aim?
2. Could a constraint be mistaken for the objective?
3. Is there a single piece of decisive evidence that would materially change the next decision?
4. Does every primary request have a verifier-decidable acceptance condition?

### 5.6 EPS semantics

An EPS is an executable realization of DIC plan nodes for one attempt. *Main authors the initial EPS/prompt graph* together with the Phase handoff. It records plan-node identity, dependencies, bounded prompt bodies, intended executor role, parallel groups, subagent flags, evidence requirements, and stop conditions. Before dispatch, Orchestrator validates this graph and compiles the prompt wording/tool-facing details for the actual model and execution surface. Multiple EPS attempts may realize the same unchanged DIC after new evidence arrives.

This separation is essential for local recovery. If a coding prompt is defective, or one implementation attempt fails, the scientific contract should not be rewritten as though the failed execution never existed. The Main-authored initial EPS remains provenance; Orchestrator may create a bounded superseding repair EPS under the same DIC.

### 5.7 Model-tuned prompt compilation

Generic templates are scaffolds, not final prompts. Before dispatching a model-executed EPS node, Orchestrator resolves the actual execution model and surface and applies the appropriate prompt guidance. The conceptual compiler is:

$$\begin{aligned}\text{DIC} + \text{current evidence} &\rightarrow \text{PROMPTING\_CORE}\\ &\rightarrow \text{provider/model overlay}\\ &\rightarrow \text{surface overlay}\\ &\rightarrow \text{bounded execution prompt.}\end{aligned}$$

The final prompt should state one primary outcome, success/evidence criteria, relevant state/dependencies, hard constraints, authority boundaries, outputs, checks, repair/stop behavior, and return contract. Duplicated or irrelevant instructions introduced by composition are removed before dispatch.

### 5.8 Coding-agent Orchestrator protocol

The coding-agent Orchestrator executes the whole Phase. Its normal loop is:

1. recover root, Phase, DIC, and current EPS/evidence state;
2. validate authority and readiness;
3. identify dependency-ready DIC plan nodes;
4. choose direct execution versus bounded subagent decomposition;
5. tune prompts to actual model/surface;
6. execute independent branches concurrently where write ownership and interfaces permit;
7. integrate returns at declared merge nodes;
8. reject incomplete evidence or create bounded repair EPS attempts;
9. dispatch verification at declared boundaries;
10. return material replan/escalation to Main rather than silently changing the DIC;
11. produce a durable handoff at Phase closure.

### 5.9 Bounded Workers and subagents

A Worker/subagent receives one complete bounded task, relevant files, allowed/forbidden scope, evidence requirements, and return contract. It does not gain Phase authority by being capable of additional work. Sibling write ownership should be disjoint unless a parent merge procedure is explicit. Independent read-only analysis can be parallelized more aggressively.

The parent remains responsible for integration. Worker completion is not Orchestrator integration; integration is not verification; verification is not human approval for external consequences.

### 5.10 Research, Theory, and Experiment integration

A Main-led research programme may follow:

```
Main question
  -> Research (if external evidence can change the decision)
  -> Main reconciliation
  -> Theory and/or Experiment specialist work
  -> Main whole-Phase DIC + initial EPS/prompt graph
  -> coding-agent Orchestrator validation / execution
  -> evidence
  -> Main reconciliation / closure / next Phase
```

Theory and Experiment may operate in parallel when their evidence is genuinely separable. Research may be requested by Main or by a specialist workstream, but any binding change to method, protocol, claim, or Phase meaning returns through Main.

### 5.11 Verification outcomes

Verification is a control signal rather than a terminal score. The canonical outcomes are:

**accept** current evidence satisfies the acceptance predicate;

**repair** the DIC remains valid but execution/evidence within its boundary must be corrected;

**replan** the current route/contract cannot honestly deliver and a higher-level plan change is required;

**escalate** human/material authority is required.

This four-way distinction prevents the common failure in which every negative result is treated as another local retry.

### 5.12 Evidence and recovery

Actions bind to evidence paths and version identities. If a repaired source changes declared keys, only the declared dynamic impact cone must be invalidated under the locality assumptions formalized later. Unaffected evidence is retained rather than discarded merely because another branch failed. Conversely, hidden dependencies invalidate the guarantee; incomplete declarations are therefore a scientific and engineering risk, not merely a bookkeeping defect.

### 5.13 Human-facing Main status

Main's visible status should remain much smaller than the runtime graph. At Phase planning/reconciliation it should normally expose only:

| Field | Human-facing meaning |
|---|---|
| Plan / Phase | current programme milestone |
| Primary Aim | central result the Phase exists to establish |
| Core workstreams | whether Main, Theory, Experiment are active |
| Research | not needed / requested / returned / integrated |
| DIC | one active DIC; Core three views complete/incomplete |
| DIC plan | concise durable sequence/parallelism summary |
| Initial EPS | Main-authored first executable prompt graph; ready/not ready |
| Evidence target | decisive evidence needed for closure |
| Human gate | none, or the exact material decision required |

**Table 4:** Compact Main-facing status.

The human should not be asked to monitor repair-EPS counts or worker logs unless those details become material evidence; Main should nevertheless be able to show that the initial EPS/prompt graph exists and is ready for handoff.

### 5.14 Current-state authority

Package state overrides chat memory. A fresh session should reconstruct the project by reading canonical files rather than relying on the previous conversation's narrative. The development distribution similarly separates current truth, future work, and history: `CURRENT_STATE.md` describes what is true now; `ROADMAP.md` describes future work; `CHANGELOG.md` records history.

## 6 Decision Rationale and Attention Economics

### 6.1 Why Main is persistent chat

Persistent chat is well suited to semantic continuity, clarification, synthesis, and material judgement. Main benefits from seeing the evolution of the user's objective but should not accumulate noisy terminal histories. Repository state remains the authority, so a new Main can recover if the original chat becomes unavailable or overloaded.

### 6.2 Why the Orchestrator is specifically a coding-agent surface

Repository execution has different requirements from semantic planning: file discovery, shell commands, patches, tests, artifact creation, environment feedback, and repeated local repair. Current coding-agent surfaces are designed for exactly these loops. Treating Orchestrator as a coding-agent session rather than another general planning chat reduces role ambiguity and makes the machine/human boundary legible.

### 6.3 Why Research is explicit but not a permanent role

External research is load-bearing only when it can change a decision. Requiring a standing Researcher for every Phase creates ceremony and context overhead; hiding research entirely makes it easy to forget that external evidence is available. The compromise is a first-class capability with an explicit Main decision gate. The returned report is advisory until Main records the binding implication.

### 6.4 Why one DIC rather than sequential contracts

Several sequential DICs inside one Phase would blur the distinction between durable meaning and execution steps. The single multi-view DIC provides one semantic anchor; its `PLAN.md` can still contain arbitrarily rich sequence, parallelism, branch, and merge structure. This is easier for humans to review and makes it possible to distinguish repair of execution from change of intent.

### 6.5 Why the execution graph lives in DIC/PLAN.md

The execution graph is part of what Main is planning: which scientific/technical work depends on what, what may run in parallel, what must merge before claims can advance, and where verification belongs. Hiding this entirely inside EPS would make the durable contract unable to explain the intended route. Conversely, storing actual prompt text and every runtime retry in `PLAN.md` would make the DIC unstable. The durable graph therefore names semantic plan nodes; EPS records their current operational realization.

### 6.6 Why Main authors the initial EPS and why EPS remains disposable

Execution conditions change: repositories evolve, tools fail, model availability changes, one branch returns negative evidence, or a prompt proves defective. Regenerating EPS lets the runtime adapt without rewriting history or weakening the approved Primary Aim. Versioned supersession also preserves provenance for later audit.

### 6.7 Why prompts are smaller than phases

A Phase may contain theory, literature, implementation, experiments, statistics, and paper changes. A single executor prompt spanning all of them is difficult to retry, hard to verify, and likely to mix evidence classes. Bounded prompts create local failure semantics and allow safe parallelism. However, excessive micro-prompting can waste context and integration effort. The split criterion is semantic: dependency, evidence, expertise, write ownership, or failure boundary—not individual shell commands.

### 6.8 Why subagent use is explicit

Subagent parallelism can reduce critical-path time and isolate contexts, but it can also multiply inconsistent interpretations and write conflicts. The Orchestrator must therefore actively consider

subagents while retaining a bias toward direct execution for simple sequential work. Every subagent must have a parent integration point and bounded authority.

### 6.9 Why DICs should become more readable as execution grows

A common anti-pattern is proportional reporting: twice as many agent actions produce twice as much human-facing prose. That makes human attention scale with machine activity. AI Native Workflow instead treats reports as indexes into evidence and DICs as compressed decision surfaces. Raw artifacts remain available, but the human-facing layer emphasizes decisions, decisive evidence, unresolved questions, and material exceptions.

### 6.10 Why the Primary Aim is explicit

Long AI-assisted research can become surprisingly flat. Every caveat, alternative, side theorem, and benchmark may look locally valuable. Explicit salience prevents this breadth from erasing the project's main scientific question. The Primary Aim also provides a reference for deciding whether optional work should continue: an extension that neither strengthens nor falsifies the central claim is a candidate for deferral.

### 6.11 Why defensibility is a constraint rather than the objective

Adversarial review is useful, but repeated hostile audits can create constraint–objective inversion. The correct optimization problem is closer to "find the strongest correct result subject to evidence and claim constraints" than "minimize all possible criticism." The workflow keeps correctness, safety, and evidence as hard boundaries while allowing ambition inside them.

### 6.12 Why human attention, not tokens, is the principal coordination quantity

Token counts and model calls measure machine resource use, not the researcher's supervisory burden. Wall-clock time conflates useful parallelism, queueing, model latency, and human waiting. Human attention is closer to the scarce resource the workflow is designed to reallocate. Evaluation should therefore record intervention classes, durations where feasible, manual routing/context-transfer events, reconstruction episodes, and the scientific materiality of interventions alongside conventional compute metrics.

### 6.13 Why attention reduction is not automatically good

A system can reduce human attention by hiding problems. The attention objective is therefore constrained by verified progress and false-acceptance outcomes. A workflow that requires little supervision but produces unsupported research should score poorly. The denominator in $\rho_H$ is verified progress rather than raw activity or number of generated artifacts.

### 6.14 Why concise reports do not imply missing evidence

The detailed evidence belongs in logs, artifacts, results, research records, verification outputs, and experiment manifests. Human reports summarize what changed and point to decisive evidence. This mirrors a broader separation between data plane and control plane: machine evidence can be large while the human control surface remains small.

# 7 Operational Use of AI Native Workflow

## 7.1 Adoption paths

Researchers may adopt the workflow at three levels:

1. **Use the full runtime**: copy `.ai-workflow/` into the repository and follow the Main/Orchestrator path.
2. **Adapt selected architecture**: keep the existing tool stack while adding explicit separation between Phase, DIC, and EPS, Research reconciliation, or verification gates.
3. **Adopt individual mechanisms**: for example one multi-view DIC per research milestone, dependency-aware repair, Primary Aim/salience labels, or model-tuned prompt compilation.

The workflow is a reference operating model, not a requirement to abandon existing project conventions.

## 7.2 Starting from an existing informal workflow

A common starting point is already something like:

```
ChatGPT / Claude persistent chat     -> planning and interpretation
Deep Research / browser              -> literature and novelty
Codex / Claude Code                  -> implementation and experiments
another chat                         -> analysis and paper writing
```

The upgrade is to externalize state and ownership. The researcher starts one persistent Main session, gives it the repository with `.ai-workflow/`, and asks it to recover current state rather than reconstructing the project manually. Main then proposes the next whole Phase and its DIC.

## 7.3 Main session protocol

At the start of a Main session:

1. read `.ai-workflow/docs/CORE.md` and `MAIN.md`;
2. inspect project-local authority files and runtime state;
3. read the active Plan, Phase, DIC Core views, prior verified handoff, and unresolved material decisions;
4. preserve exact user input separately from Main's interpretation;
5. identify the Primary Aim and detect any defensive drift or salience flattening;
6. decide whether external Research, Theory, or Experiment work can materially improve the next decision;
7. author or revise the single whole-Phase DIC;
8. present the compact Phase checklist for human correction;
9. hand the whole Phase to the coding-agent Orchestrator when the DIC is ready.

Main should not present a second chat-level execution prompt graph: the graph already belongs in `DIC/PLAN.md`.

## 7.4 Research pass

A Research request should name the exact decision it informs. Useful forms include novelty audit, benchmark/regime search, official specification lookup, method comparison, or literature-based falsification. The return should include an executive conclusion, source hierarchy, competing

evidence, uncertainty, one ranked recommendation with fallback, and exact implications for DIC planning. Research output is not automatically binding and must not claim an implementation or experiment has occurred.

### 7.5 Theory workstream

Theory receives a bounded scientific question and the evidence needed to reason about it. For material claims, separate positive derivation, hostile audit, and novelty comparison where useful. Theory returns the strongest correct central statement, assumptions, counterexamples, and which side results are actually load-bearing. It should not turn every possible lemma into a permanent research branch.

### 7.6 Experiment workstream

Experiment freezes estimands, controls, benchmark design, seed/data namespaces, evidence ceilings, and analysis plans before confirmatory runs. Screening, development, pilot, locked, and verified evidence are separated. Experiment may delegate implementation through the Orchestrator but should not revise a frozen protocol after observing results without Main-level replanning.

### 7.7 Constructing the DIC

A good DIC can be reviewed by a researcher without reading the execution history. The Core views should answer:

**README.** What Phase is this? What is the Primary Aim? What has already been established? What is in scope, optional, or explicitly out of scope? What authority and human gates apply?

**REQUESTS.** What observable results must exist before the Phase can close? Which requests are primary versus supporting? What evidence class is required? What exact conditions can a verifier decide?

**PLAN.** What nodes must execute? Which can run in parallel? Which files or conceptual interfaces are shared? Where are merge/integration barriers? What evidence gates block downstream claims? What repair branches are anticipated?

### 7.8 Default project-local Phase filesystem

Project-specific Phase material is stored at repository root under `phases/<phase_id>/`, rather than inside the reusable `.ai-workflow/` runtime. The default scaffold contains the DIC, Main-authored EPS artifacts, compact reports, structured results where appropriate, an evidence manifest, artifacts, and logs. In particular, `reports/PHASE_SUMMARY.md` is the current human-attention surface; `reports/FINAL_HANDOFF.md` is required at terminal handoff; `results/KEY_RESULTS.json` is required when key findings are naturally structured; and `results/KEY_RESULTS.` is added when a tabular representation is natural. The evidence manifest indexes decisive machine-readable evidence without forcing raw logs into human reports.

### 7.9 Example research Phase

Suppose the research question is whether a proposed compatibility mechanism genuinely improves attention fine-tuning beyond ordinary capacity. A DIC might state:

```
Primary Aim:
  Determine whether the mechanism has stable benefit beyond matched Q/K capacity.

Parallel branches:
  A. literature/novelty audit
  B. theory/counterexample analysis
  C. frozen matched-capacity experiment

Merge:
  Main reconciliation after A/B/C

Evidence gate:
  no venue-facing positive claim unless the matched control and statistical
  analysis satisfy the frozen acceptance criteria.
```

This plan is durable. Main authors the first EPS that realizes its branches; the Orchestrator may later issue bounded repair EPS attempts if implementation or runs fail without invalidating the DIC.

### 7.10 Orchestrator start and readiness

The Orchestrator should be initialized with the whole Phase/DIC and relevant project files, not a vague instruction to "continue the project." It first validates repository root, authority, dependency readiness, tools, the Main-authored initial EPS, and current evidence. It then compiles the EPS prompts for the actual execution surface and schedules the ready nodes.

### 7.11 Prompt compilation and model routing

For each EPS node, Orchestrator resolves whether it will execute directly or through another model/subagent. It applies the current prompting core plus the provider/surface overlay. For example, Codex execution uses the OpenAI/GPT routing and Codex guidance; Claude Code uses the Claude execution guide. The final prompt is then audited for one primary outcome, correct dependencies, allowed paths, evidence, tests, and stop conditions.

### 7.12 Parallelism protocol

Parallelize only when independence is real. Safe examples include separate read-only literature scans, file-disjoint modules, independent experiment seeds under a frozen protocol, or positive/hostile proof attacks returning reports. Unsafe examples include multiple siblings editing the same canonical manuscript or changing a shared schema without an integration barrier.

A useful ownership record includes plan node, executor, read paths, write paths, dependencies, evidence path, and merge node. The Orchestrator may dynamically reduce parallelism when unexpected coupling appears.

### 7.13 Integration

Integration is a separate step from worker completion. The parent checks interface compatibility, evidence completeness, stale dependencies, unexpected scope changes, and repository cleanliness. If the return is locally defective but the DIC remains valid, create a repair EPS. If the finding

changes the scientific method, acceptance, authority, or Primary Aim, return to Main for replan/escalation.

### 7.14 Verification and closure

A Phase closes only after acceptance-bearing requests have current evidence and the verifier outcome is `accept`. Failed or negative branches remain in the record. The final handoff should identify the DIC, terminal EPS/run identities, decisive evidence, unresolved caveats, release/submission state, and the compact state needed for the next Phase.

### 7.15 Recovery after failure

If a source node changes output keys after repair, compute the declared dependency cone and invalidate only affected downstream evidence under the locality assumptions. The system must not infer safety from convenience; undeclared mutable dependencies make local retention unsafe. A replan is appropriate when the repair would alter the scientific contract rather than only the operational realization.

### 7.16 Fresh-session recovery

A fresh Main or Orchestrator should be able to reconstruct the project from repository state. Chat memory can accelerate understanding but is not the authority. Recovery should validate Plan/Phase/DIC/EPS lineage, required DIC views, versioned bindings, and evidence pointers, then surface warnings/errors rather than silently repairing inconsistent state.

### 7.17 Human attention and checkpointing in practice

Record human interventions when feasible using a small taxonomy: semantic clarification; scientific/methodological decision; manual routing/context transfer; execution troubleshooting; evidence interpretation; claim/release decision; security/external approval. Count events, estimate duration where reasonable, and distinguish interventions that materially changed the science from routine coordination. The goal is not to eliminate human involvement; it is to move routine interventions downward so that the remaining human attention is more decision-dense.

### 7.18 Non-research technical work

The same runtime can support engineering, documentation, quantitative analysis, or product work. Research-specific Theory/Experiment workstreams may be inactive. Small bounded maintenance can be handled directly by the Orchestrator or a narrow worker without teaching the Education-track audience a separate mode taxonomy. The invariant remains the same: use only as much process as the task justifies, preserve evidence, and escalate when objective/authority changes.

### 7.19 External, destructive, and public actions

Submission, release, purchases, credentials, private external systems, permission expansion, and destructive actions remain explicit human gates. An agent's ability to perform an action is not evidence of authorization. Where possible, use dry runs, exact targets, least privilege, backups/snapshots, and post-action verification.

### 7.20 Tracking the workflow project itself

The development distribution should not require reading several competing status files. The canonical root tracking files are:

`00_READ_FIRST.md` very short authority/index and current version;

`CURRENT_STATE.md` what is true now;

`ROADMAP.md` future phases, deferred work, deadlines, and human decisions;

`README.md` stable overview and distribution/runtime boundary;

`QUICKSTART.md` one adoption path;

`VERSIONING.md` version policy;

`CHANGELOG.md` cumulative history;

`PACKAGE_MANIFEST.json` file inventory and hashes;

`PACKAGE_HANDOFF.json` machine-readable recovery state.

Older Main reports, next-run notes, duplicate Quick Starts, and per-version changelogs are preserved under `history/` rather than competing with current truth.

## 8 Formal Model and Theory: Orientation

The previous sections explain the operating architecture. This part states the formal objects and strongest currently supported structural results. The goal is not to decorate the workflow with mathematics; it is to expose assumptions that prose can otherwise hide. In particular, hierarchy alone does not guarantee intent preservation, local repair is unsafe under hidden dependencies, and verifier confidence cannot overcome evidence that aliases success and failure.

## 9 Formal Hierarchical Execution Model

### 9.1 Levels and roles

The canonical levels are shown in Table 5.

Main owns overall authority, architecture, integration, state, and publication. Theory owns formal reasoning, literature and novelty analysis, propositions, and counterexamples. Experiment owns benchmark design, implementation, statistics, and empirical evidence. Search and research are capabilities available to Theory and Experiment. A workstream may delegate bounded actions, but it retains its DIC and integration responsibility.

### 9.2 Plans and phases

**Definition 9.1** (Plan)**.** *A Plan is a tuple $P = (V, E, \prec, \Gamma, h)$ where $(V, E)$ is a directed acyclic graph of phases, $\prec$ is a declared execution order compatible with $E$, $\Gamma$ is the governance policy, and $h$ is the periodic human-checkpoint interval. Each phase $v \in V$ has an owner, objective, dependency set, materiality class, DIC pointer, status, and evidence index.*

A Plan is the normal human approval object. A Phase is a machine milestone, not ordinarily a separate human approval request. The runtime permits automatic progression to a dependency-ready ordinary phase until a checkpoint becomes due or a material escalation opens.

| Level | Lifetime | Semantic responsibility | Normal authority |
|---|---|---|---|
| Goal | programme | global outcome, constraints, authority | human + Main |
| Plan | several phases | strategic DAG and checkpoints | human approval, Main ownership |
| Phase | milestone | coherent research milestone | Main, with Theory/Experiment support |
| DIC | phase/repair-stable | Primary Aim, acceptance, evidence, durable execution graph | Main |
| EPS | disposable | JIT realization of ready DIC plan nodes | coding-agent Orchestrator |
| Prompt/Run | attempt-local | bounded model execution and context | Orchestrator/Worker/subagent |
| Action | primitive | tool use, edit, search, experiment, test | executor/tool |

**Table 5:** The hierarchy separates durable semantics from disposable operations.

### 9.3 Durable intent contracts

**Definition 9.2** (DIC)**.** *A durable intent contract for phase $v$ is*

$$C_v = (D_v, I_v, A_v, W_v, \mathcal{U}_v, \mathcal{X}_v, \mathcal{R}_v),$$

*where $D_v \subseteq \mathcal{S}$ is a precondition domain, $I_v \subseteq \mathcal{T}$ is a trace invariant, $A_v \subseteq \mathcal{S}$ is an acceptable post-state set, $W_v : \mathcal{S} \times \mathcal{E}^* \to \{0, 1\}$ is an evidence predicate, $\mathcal{U}_v$ is the authority/input set, $\mathcal{X}_v$ is the escalation set, and $\mathcal{R}_v$ is the declared dependency/read footprint.*

The runtime materializes one DIC package per Phase rather than a sequence of stepwise DICs. The DIC is deliberately dual-use: a machine contract and a human-intelligibility projection. Required views are `README.md` (Primary Aim, priority/salience structure, authority, and compact orientation), `REQUESTS.md` (prioritized outcomes and acceptance requirements), and `PLAN.md` (the durable execution graph, including prompt identifiers, dependencies, parallel subagent batches, merge points, and conditional repair/review branches). Optional `ARCHITECTURE.md`, `TEST_MATRIX.md`, and user-defined flat Markdown views expose additional angles on the same contract. A machine `MANIFEST.json` indexes these views and content hashes. A DIC survives local repair. If the subgoal, acceptance meaning, frozen protocol, or authority changes materially, the correct operation is replan or escalate rather than silently mutating the contract.

### 9.4 EPS, prompt/runs, and actions

The DIC execution plan is durable, while an EPS is a disposable just-in-time instantiation of selected plan nodes against the current repository state, model/context, attempt, and repair history. An EPS may name prompt/run IDs, dependencies, parallel groups, subagent delegations, expected outputs, local checks, and stop conditions, and it may materialize executable prompt files. Ordinary failure or retry therefore creates a new EPS/run instance without changing the DIC unless the Phase contract or durable execution design itself changes. An Action record links to an EPS and, when applicable, a prompt/run node, while preserving DIC, Phase, Plan, result, and evidence lineage.

# 10 Intent and Contract Refinement

## 10.1 Trace semantics

Associate each object $X$ in the hierarchy with an admitted trace set $[\![X]\!] \subseteq \mathcal{T}_X$. Adjacent levels may use different state descriptions, so let $\rho_{XY} : \mathcal{T}_X \to \mathcal{T}_Y$ be a declared abstraction map.

**Definition 10.1** (Semantic refinement)**.** *$X$ refines $Y$, written $X \preceq_\rho Y$, if*

$$\rho_{XY}([\![X]\!]) \subseteq [\![Y]\!].$$

This definition makes intent preservation a semantic inclusion claim, not a similarity judgment between prompt strings. In a practical package, refinement is supported by contract predicates, evidence, tests, and review; it is rarely decided by a complete semantic oracle.

**Proposition 10.2** (Refinement-chain preservation)**.** *If $X_0 \preceq X_1 \preceq \cdots \preceq X_m$ and the abstraction maps compose on admitted traces, then $X_0 \preceq X_m$.*

*Proof.* Repeated image inclusion gives $\rho_{0m}([\![X_0]\!]) \subseteq \rho_{1m}([\![X_1]\!]) \subseteq [\![X_m]\!]$. □

The proposition is a consequence of the definition. It does not show that a model-generated child actually refines its parent; that requires a sound certificate or verifier.

## 10.2 Acceptance predicates and acceptable-state sets

For a completed phase trace $\tau_v$ ending in state $s'$, define

$$\mathrm{Acc}_{C_v}(\tau_v, e_v) = \mathbf{1}\{s' \in A_v,\ \tau_v \in I_v,\ W_v(s', e_v) = 1\}.$$

Acceptance separates the desired state from evidence that warrants believing the state was reached. A unit test may be strong evidence for a deterministic software property and weak evidence for a broad scientific claim. The DIC must state the connection.

## 10.3 Compositional satisfaction

For each phase $v$, let $F_v$ be its implemented transition relation. Let $\mathrm{Pred}(v)$ be its predecessors. Write $\mathsf{Pre}_v$ and $\mathsf{Post}_v$ for its pre/post predicates and $\mathsf{Frame}_v$ for keys it promises not to alter.

**Assumption 10.3** (Contract soundness)**.** *For every phase $v$, every admitted execution satisfies the Hoare-style relation $\{\mathsf{Pre}_v \wedge I\}\, F_v\, \{\mathsf{Post}_v \wedge I\}$, and its evidence predicate is sound for the asserted postcondition.*

**Assumption 10.4** (Edge compatibility)**.** *For each non-source $v$, the conjunction of relevant predecessor postconditions implies $\mathsf{Pre}_v$. Concurrent or unordered branches either have disjoint writes or satisfy an explicit commutativity/noninterference condition.*

**Theorem 10.5** (Compositional satisfaction)**.** *For an acyclic Plan satisfying contract soundness and edge compatibility, if the initial state satisfies all source preconditions and the conjunction of terminal postconditions implies the global goal $G$, then every admitted complete topological execution terminates in a state satisfying $G$ and the global invariant $I$.*

*Proof.* Proceed by induction over a topological order. Source preconditions hold initially. At step $v$, all predecessors have executed; edge compatibility and noninterference imply $\mathsf{Pre}_v$, while the induction hypothesis gives $I$. Contract soundness yields $\mathsf{Post}_v$ and preserves $I$. After all nodes, all terminal postconditions and $I$ hold; the terminal implication gives $G$. □

The assumptions are substantial. Hidden dependencies, unsound evidence, stochastic tools outside the contract, or incompatible branches invalidate the conclusion. The theorem is a precise restatement of contract composition in this setting, not a claim that LLM agents automatically satisfy it [100].

# 11 Verification and Replanning

## 11.1 Closed-loop operator

Verification consumes the DIC, current state, emitted evidence, and relevant dependency versions. It returns

$$\mathcal{V}(C_v, s, e) \in \{\mathsf{accept}, \mathsf{repair}, \mathsf{replan}, \mathsf{escalate}\}.$$

Accept closes the DIC. Repair retains the DIC and regenerates EPS. Replan changes the decomposition or contract and therefore crosses a governance boundary. Escalate requests authority unavailable to the active workstream.

Local verification is automatic where a tool or predicate can check the action. Independent review is reserved for material evidence, claim freeze, release, or submission boundaries. Independence means a fresh context and no authorship of the audited artifact; merely repeating the same prompt does not remove shared blind spots.

## 11.2 Imperfect verification

Let a fault remain present at successive gates. Define $M_i$ as the event that it survives gate $i$ undetected.

**Assumption 11.1** (Conditional miss bound)**.** *For some $0 \le \alpha < 1$, $\Pr(M_i \mid M_1 \cap \cdots \cap M_{i-1}) \le \alpha$ at every gate at which the fault is observable.*

**Theorem 11.2** (Survival and detection bound)**.** *Under the conditional miss bound,*

$$\Pr(M_1 \cap \cdots \cap M_k) \le \alpha^k.$$

*If $N$ is the number of gate intervals through which the fault persists before detection, then $\mathbb{E}[N] \le (1-\alpha)^{-1}$.*

*Proof.* The chain rule and the conditional bounds give the product bound. The tail-sum formula gives $\mathbb{E}[N] = \sum_{k\ge 0} \Pr(N > k) \le \sum_{k\ge 0} \alpha^k = (1-\alpha)^{-1}$. □

No independence assumption is needed beyond the stated conditional bound. The bound is useless when the fault is not represented in the evidence channel, a limitation formalized in Definition 17.1.

## 11.3 Repair versus replan

A repair changes operational means while preserving the DIC's semantics and authority. A replan changes at least one parent-level obligation, dependency, acceptance meaning, or frozen protocol. This distinction is auditable: a repair produces a new EPS under the same DIC hash; a replan changes a DIC or Plan revision and opens a checkpoint or escalation event.

# 12 Dependency Locality and Failure Propagation

## 12.1 Declared footprints

Model state as a key-value map $s : \mathcal{K} \to \mathcal{V}$. Each phase $v$ declares read set $R_v \subseteq \mathcal{K}$ and write set $W_v \subseteq \mathcal{K}$. Edges include every possible data dependence: if $W_u \cap R_v \neq \varnothing$, then $u \to v$ unless an equivalent versioned external dependency is declared.

After a repair changes keys $\Delta \subseteq \mathcal{K}$, define the direct-reader seed

$$B(\Delta) = \{v : R_v \cap \Delta \neq \varnothing\}$$

and dynamic impact cone

$$\mathcal{I}(\Delta) = \mathrm{Reach}_G(B(\Delta)),$$

including the seed.

**Assumption 12.1** (Local deterministic-or-reverified semantics)**.** *A phase's output and evidence validity depend only on its declared read values, declared predecessor outputs, explicit tool/version parameters, and fresh randomness recorded in evidence. There is no undeclared mutable shared state. Evidence is bound to input and artifact versions.*

**Theorem 12.2** (Dependency-local recovery)**.** *Under complete dependency edges and local deterministic-or-reverified semantics, after a repair that changes only $\Delta$, retaining completed phase outputs and evidence for every $v \notin \mathcal{I}(\Delta)$ preserves their contract validity. Re-execution need be considered only inside $\mathcal{I}(\Delta)$.*

*Proof.* Take a topological order of nodes outside the cone. A node $v \notin \mathcal{I}(\Delta)$ does not directly read a changed key. Nor can any predecessor output on which it depends have changed: otherwise a changed predecessor would reach $v$, placing $v$ in the cone. By induction, all declared inputs, tool/version parameters, and predecessor outputs of $v$ are unchanged. Local semantics therefore preserve its output distribution under recorded randomness, or fresh reverification certifies it, and version-bound evidence remains valid. □

**Theorem 12.3** (Worst-case tightness)**.** *Without additional semantic restrictions, every node in $\mathcal{I}(\Delta)$ can be necessary to invalidate: for any such node, there exists a set of phase functions consistent with the declared graph for which changing $\Delta$ changes that node's accepted output. Hence no strict subset of the impact cone is uniformly safe over the declared-footprint model.*

*Proof.* For a direct reader, choose its output to copy a changed bit. Along any path from that reader to a target descendant, choose each phase to copy the predecessor bit and let its acceptance predicate bind to the previous version. The change propagates to the target and invalidates its old evidence. Thus each reachable node is necessary in some admissible realization. □

This result explains both the value and fragility of local repair. Complete dependency declarations can reduce rework from a full rerun to $\sum_{v \in \mathcal{I}(\Delta)} c_v$; one hidden edge can make that guarantee false. The structure parallels dependency graphs, incremental build systems, and self-adjusting computation [101, 102, 103].

## 12.2 Propagation bounds

A fault originating at node $u$ can affect at most $|\,\mathrm{Desc}(u)| + 1$ phases and in the worst case affects all of them. If the descendant subgraph has branching at most $b$ and depth at most $d$, then

$$|\,\mathrm{Desc}(u)| + 1 \leq \sum_{i=0}^{d} b^i,$$

with the usual $d + 1$ limit when $b = 1$. Verification gates do not alter this structural cone; they reduce how much of it executes before detection and which outputs remain trusted.

# 13 Context and Persistent State

## 13.1 Package as authority

Chat history is not a durable database. The package therefore stores a small authoritative index and immutable or versioned semantic records. A fresh workstream reads the approved goal, Plan, active Phase, complete multi-view DIC, EPS/prompt/action lineage, compact reports surface, evidence index, checkpoints, and escalations from the package. Generated narrative handoffs are useful views, not competing authorities.

## 13.2 Context projection

Let $\Pi_v(s)$ be the context projection supplied to the executor for phase $v$, and let $\pi_v(a \mid s)$ be the ideal phase action kernel.

**Proposition 13.1** (Context-local execution)**.** *Omitting state outside $\Pi_v(s)$ leaves phase behavior unchanged if*

$$\pi_v(a \mid s) = \pi_v(a \mid \Pi_v(s))$$

*for all admissible $s, a$, all acceptance predicates and escalation conditions are measurable from the projection plus resolvable evidence references, and omitted state cannot mutate through a hidden channel. If two admissible states share a projection but require different action distributions or acceptance decisions, the projection is insufficient.*

*Proof.* The sufficient direction follows by substitution in the action and verification kernels. For necessity, take states $s, s'$ with $\Pi_v(s) = \Pi_v(s')$ but different required behavior; any projection-only executor must act identically on them and is wrong on at least one. □

Externalized state reduces context load only when the projection is semantically sufficient. It is not made safe by storing more files whose relevance cannot be recovered. MemGPT and hierarchical memory systems motivate externalization and working-memory control [83, 84]; the DIC specifies what must be projected for one bounded phase.

## 13.3 DIC as a human-attention projection

The same projection problem applies to the researcher. Let $\Pi_H(s)$ denote the human-facing Phase state. Requiring the human to read every prompt, transcript, terminal log, failed branch, and test output makes supervisory burden grow with machine activity. The DIC instead exposes a compact current projection: Primary Aim, prioritized requests, durable topology, decisive evidence pointers, unresolved material questions, and authority boundaries. Detailed execution remains addressable by path when a decision requires it. The design principle is therefore *increasing machine complexity should not imply increasing human reconstruction complexity.* A compact DIC is not safe merely because it is short; it must remain sufficient for the material decisions delegated to the human.

# 14 Human Oversight Across Timescales

## 14.1 Human-attention intensity

Let $\mathcal{H}(\tau)$ be the human-attention events in run $\tau$: approvals, material adjudications, context reconstruction, manual routing, evidence inspection, or other supervisory interventions. Let event $j$ have duration $t_j$ and a prespecified attention weight $w_j \geq 0$. Define

$$A_H(\tau) = \sum_{j \in \mathcal{H}(\tau)} w_j t_j.$$

When detailed timing is unavailable, event counts by intervention class are a lower-fidelity proxy. Let verified research progress be

$$P_V(\tau) = \sum_{v \in V} q_v z_v,$$

where $q_v > 0$ is a prespecified milestone weight and $z_v \in \{0, 1\}$ indicates that the milestone is accepted against current evidence. The *human-attention intensity*

$$\rho_H(\tau) = \frac{A_H(\tau)}{P_V(\tau) + \varepsilon}$$

measures human supervisory attention per unit of verified progress. Its reciprocal is an attention-efficiency score. Because progress units and weights are project-specific, $\rho_H$ is meaningful primarily for matched tasks or within a frozen benchmark definition; it is not a universal scalar of research quality.

The workflow targets lower $\rho_H$ by moving routine routing, state reconstruction, local repair, and evidence bookkeeping below the human boundary while preserving escalation for high-value judgement. This is an empirical objective, not a demonstrated result. A badly designed hierarchy can increase both $A_H$ and error rates.

## 14.2 Governance complexity

Suppose a Plan has $N$ ordinary completed phases, periodic checkpoint interval $h$, and $M$ material escalations. Beyond initial Plan approval, the number of mandatory human governance events is at most

$$\left\lfloor \frac{N}{h} \right\rfloor + M$$

(up to a chosen final release review). This is a counting invariant of the policy, not a quality guarantee. Machine-local checks may be much more frequent. If each mandatory governance event has attention cost at most $\bar{a}$ and initial Plan review costs $a_0$, the policy-level contribution is bounded by $a_0 + \bar{a}(\lfloor N/h \rfloor + M)$; optional deep investigations and difficult escalations may exceed this bound and should be measured separately.

## 14.3 Checkpoint interval under imperfect verification

Consider a simplified stationary model. Each checkpoint costs $c_v$; a false rejection occurs with probability $\beta$ and costs $c_f$; faults arrive at rate $\lambda$ per phase; re-executing one contaminated phase costs $c_r$; a persisting observable fault is missed at a gate with conditional probability $\alpha < 1$. With interval $h$, a fault appears uniformly within its first block. Its expected contaminated span before detection is

$$\frac{h}{2} + h\frac{\alpha}{1-\alpha} = h\frac{1+\alpha}{2(1-\alpha)}.$$

Ignoring overlapping faults and edge effects, expected overhead over $n$ phases is

$$U(h) = \frac{n(c_v + \beta c_f)}{h} + \lambda n c_r h \frac{1+\alpha}{2(1-\alpha)}.$$

**Proposition 14.1** (Cost-optimal interval in the stationary model)**.** *The continuous minimizer is*

$$h^* = \sqrt{\frac{2(c_v + \beta c_f)(1-\alpha)}{\lambda c_r (1+\alpha)}}.$$

*The discrete policy uses a feasible integer near $h^*$.*

*Proof.* $U$ is strictly convex for $h > 0$. Setting its derivative to zero and solving gives the expression. □

The formula recovers the qualitative checkpoint/restart trade-off [104, 105]: expensive review favors longer intervals; frequent or costly faults favor shorter intervals; a weak verifier changes both detection delay and repair overhead. The runtime default of seven is a configurable operating prior, not an estimate of $h^*$ for every project.

# 15 Reference Implementation

## 15.1 Authority and files

The v5.6.39 reference runtime uses the Python standard library for its canonical state machine. `.ai-workflow/STATE.json` records schema version, active Plan, phase summaries, DIC identity/hash, EPS attempts, Actions, verification outcomes, and evidence pointers. Human-readable DIC views are materialized under `phases/<phase>/DIC/`; EPS and Action lineage remains indexed in canonical state and may point to materialized prompts or evidence files. Exactly one DIC is permitted per Phase, and its three Core views are required. The runtime checks DIC graph acyclicity, EPS–DIC identity/hash consistency, Action lineage, verification outcomes, and package recovery. Model-executed EPS nodes cannot record Actions until the selected model/surface and prompt-guidance profile are recorded and the prompt is marked tuned. These are implementation invariants, not semantic guarantees about the world.

## 15.2 State transitions

The principal transitions are:

| Transition | Preconditions | Result |
|---|---|---|
| create Plan | initialized package | draft Plan |
| add Phase | draft Plan | dependency-checked Phase |
| approve Plan | at least one Phase | approved authority boundary |
| start Phase | approved, dependencies complete | active Phase/DIC |
| generate EPS | DIC/PLAN exists | disposable prompt/run instantiation |
| complete Phase | active, acceptance satisfied | evidence + next phase/checkpoint |
| verify repair | same DIC remains valid | regenerate EPS |
| verify replan | semantic decomposition changes | governance event |
| escalate | material trigger | autonomous progression stops |
| recover | package exists | lineage/hash/dependency audit |

### 15.3 Compatibility

Legacy v5.6.31 Goal/context/handoff files are indexed rather than deleted. `Orchestrator` aliases Main; `Researcher` aliases Theory by default or Experiment for empirical work. A legacy single prompt can be migrated as a one-Phase Plan. The compatibility layer is deliberately subordinate to the canonical state to prevent multiple status files from claiming authority.

### 15.4 Conformance coverage

The test suite covers one Plan authorizing several phases, autonomous ordinary progression, periodic checkpoint triggering, immediate material escalation, reconstructable lineage, Main/Theory/Experiment ownership, package-only recovery, and legacy compatibility, plus dependency blocking and verification outcomes. These are engineering tests. They do not measure language-agent task success.

## 16 Detailed Reference Implementation

### 16.1 Runtime layout

The machine-installable root contains deterministic state/tooling and compact operational guidance:

```
.ai-workflow/
  VERSION
  STATE.json
  config.toml
  aw.py
  aw/
    cli.py
    store.py
    impact.py
    demo.py
    templates/
  docs/
    CORE.md
    MAIN.md
    THEORY.md
    EXPERIMENT.md
    RESEARCH.md
    ORCHESTRATOR.md
    WORKER.md
    VERIFIER.md
    PROMPTING_CORE.md
    provider / execution overlays
  guides/
    DETAILED_WORKFLOW_GUIDE.md
    WORKFLOW_CLI.md
    PROMPTING_AND_MODELS.md
    SECURITY_AND_EXTERNAL_ACTIONS.md
    INSTALL_TO_REPOSITORY.md
  tests/
```

Compact `docs/` files control ordinary execution. Long-form `guides/` are on-demand human/-maintainer references and are not loaded wholesale into normal Worker prompts.

### 16.2 State-machine invariants

The runtime validates that a Phase has exactly one DIC, required Core views, and acyclic DIC plan dependencies; that each EPS binds to the correct DIC and plan-node identities; that Actions bind back to Prompt/Run, EPS, DIC, Phase, and Plan; and that closure follows an accepted verification state. Malformed lineage is an error rather than an invitation to infer missing state from chat history.

### 16.3 Prompt-tuning provenance

For model-executed nodes, runtime metadata records the target model/surface and prompt-guidance profile so an Action is not treated as a properly compiled execution if the prompt-tuning contract is absent. This does not prove the prompt is optimal; it makes the adaptation step auditable and prevents a generic template from silently becoming the final executor instruction.

### 16.4 Deterministic demonstration

The package contains a deterministic worked trace in which a seven-node DIC is first realized, a verifier records `repair` on a producer, the changed key invalidates a declared downstream cone, an independent node is retained, a second EPS re-executes only the affected scope plus repair source, and re-verification records `accept`. The trace is structural/runtime evidence, not comparative evidence of superior agent performance.

## 17 Formal Properties and Limits

### 17.1 What is guaranteed by construction

Identifier lineage, role normalization, checkpoint counters, escalation categories, atomic state replacement, and DIC hash checks are implementation invariants when callers use the canonical runtime. They are not semantic guarantees about the world.

### 17.2 Observability limit

Let $O : \mathcal{S} \to \mathcal{O}$ be the evidence observation available to a verifier and $G : \mathcal{S} \to \{0, 1\}$ the relevant goal predicate.

**Proposition 17.1** (Evidence-aliasing impossibility)**.** *If there exist $s_+, s_-$ such that $O(s_+) = O(s_-)$ but $G(s_+) = 1$ and $G(s_-) = 0$, then no verifier $V : \mathcal{O} \to \{\mathsf{accept}, \mathsf{reject}\}$ can have both zero false acceptance and zero false rejection on $\{s_+, s_-\}$.*

*Proof.* $V$ receives the same input on both states and must return the same decision. Accepting falsely accepts $s_-$; rejecting falsely rejects $s_+$. □

No amount of hierarchical decomposition overcomes evidence that fails to distinguish success from failure. The remedy is a richer environment-grounded observation, a weaker claim, or explicit residual uncertainty—not a more confident verifier prompt.

### 17.3 Open-world and adversarial limits

The theory assumes declared dependencies, stable identifiers, and evidence tied to the asserted property. Open-world side effects, prompt injection, mutable external services, and adversarial evidence can violate those assumptions. ToolEmu and AgentDojo illustrate why sandbox and security evaluation are separate requirements [96, 97]. The runtime is a control layer, not a security boundary.

### 17.4 Novelty limit

The formal ingredients have established analogues in program logics, runtime verification, build systems, and checkpointing [100, 106, 102, 105]. The report therefore claims neither theorem novelty in isolation nor implementation equivalence to a learned hierarchical world model. LeCun's world-model programme and joint-embedding predictive models are conceptual motivation for abstraction across timescales, not descriptions of this symbolic package protocol [98, 99].

## 18 Empirical Hypotheses and Evaluation Design

### 18.1 Central hypotheses

The primary empirical hypothesis is that the relative benefit of hierarchy increases with effective task horizon after matching model, task information, and inference budget. The mechanism hypothesis is that verification-triggered local repair reduces downstream propagation and rework beyond hierarchy alone and beyond spending the same calls on generic critique.

### 18.2 Conditions

The frozen candidate design contains four main conditions: C0 direct/reactive; C1 flat plan-and-execute; C2 hierarchical execution; and C3 hierarchy plus evidence-gated repair/replanning. C1+V spends C3's verifier budget on generic critique without dependency-local repair; C2-extra spends it on additional generation. These controls distinguish structure from extra tokens, calls, stronger prompts, or verifier-as-additional-inference.

### 18.3 Task factors and faults

Controlled factors are dependency depth, branching, delayed dependency distance, interacting constraints, and injected intermediate faults. Task families should have objective executable graders and pre-generated instances. Faults include stale schemas, omitted constraints, dependency-version swaps, corrupted statistics, unsupported citation pointers, and failing code paths. The injector hides fault existence and location from the agent.

### 18.4 Outcomes and analysis

Primary outcomes are end-to-end success, hard-constraint satisfaction, unsupported completion, failure-propagation size, recovery locality, detection latency, and rework. Resource measures include calls, tokens, wall time, verifier calls, and tool use. Human-supervision outcomes include attention-event counts by class, measured or logged intervention duration where feasible, context-reconstruction episodes, manual routing events, and human-attention intensity $\rho_H$ under prespecified milestone weights. The primary statistical analysis is a condition-by-effective-horizon interaction in a mixed-effects model, with paired instance contrasts and full interval estimates. A positive submission-level claim requires frozen prompts, matched budgets, at least two task families, objective graders, and completion of all primary conditions.

### 18.5 Current evidence boundary

The current reconstruction includes conformance tests and a deterministic dependency-locality fixture. They validate implementation paths and terminology only. No large confirmatory comparative study has been run, so the package does not claim empirical superiority.

# 19 Detailed Evaluation Programme

## 19.1 Evaluation question

The central empirical question is not whether a hierarchical workflow can generate more artifacts. It is whether, under matched task information and reasonably matched inference budgets, the workflow increases trustworthy end-to-end progress as effective horizon grows while reducing the human attention spent on routine coordination and recovery.

## 19.2 Primary conditions

The core comparison retains four conditions:

1. **C0 Direct/reactive**: ordinary interactive agent use without durable hierarchical state;
2. **C1 Flat plan-and-execute**: one explicit plan but no durable Phase/DIC/EPS hierarchy;
3. **C2 Hierarchical**: Goal/Plan/Phase/DIC/EPS structure without verification-triggered local recovery;
4. **C3 Full**: hierarchy plus evidence-gated accept/repair/replan/escalate and dependency-local recovery.

Matched controls can spend C3's additional verifier budget on generic critique or extra generation, distinguishing structural benefit from extra inference.

## 19.3 Representative task families

A low-cost evaluation suite should include both research-like and engineering-like tasks:

| ID | Task | Primary measurement | Seeded risk/ambiguity |
|---|---|---|---|
| R1 | Literature + implementation decision with conflicting sources | decision accuracy, citation correctness, propagation of research evidence | plausible secondary source contradicts primary source |
| R2 | Matched experiment programme with baseline, treatment, and paper table | protocol fidelity, unsupported completion, stale-artifact rate | one midstream run/schema fault |
| R3 | Theory + hostile audit + integration | theorem correctness, assumption preservation, salience of central claim | attractive but false strengthening |
| R4 | Multi-file research artifact update | dependency tracking, claim/evidence alignment, recovery locality | changed result invalidates only part of manuscript |
| E1 | Add a CLI validation rule with tests | implementation correctness, test selection, evidence | adjacent reusable rule has different semantics |
| E2 | Modify two file-disjoint components | safe parallelism, integration, human routing events | one hidden dependency disclosed in DIC |

| | | | |
|---|---|---|---|
| E3 | Ambiguous "clean up the project" request | intent preservation, authority restraint | multiple plausible meanings including deletion |
| E4 | Bounded documentation repair | scope control, prompt granularity, human attention | unrelated style issues nearby |

### 19.4 Effective-horizon factors

Vary dependency depth, branching factor, delayed dependency distance, number of interacting constraints, number of model surfaces, amount of state outside the active context, and delay between fault injection and observability. These variables should be generated independently where possible so "long horizon" is not merely synonymous with more tokens.

### 19.5 Fault classes

Inject stale schemas, omitted constraints, dependency-version swaps, corrupted statistics, unsupported citation pointers, failing code paths, contradictory DIC/request wording, an optional branch that begins to dominate the Primary Aim, and a reviewer-style instruction that encourages defensive drift. The system should not be told which fault exists.

### 19.6 Human-attention outcomes

Predefine an intervention taxonomy and measure:

- number of manual context transfers between AI surfaces;
- number of manual routing/scheduling decisions;
- context-reconstruction episodes after session changes;
- time spent reading agent status/output purely to recover project state;
- semantic/scientific interventions;
- execution troubleshooting interventions;
- claim/release/security decisions;
- total supervisory attention time where logging is feasible;
- attention-event count weighted by prespecified burden class;
- $\rho_H$ using frozen verified-progress weights.

Report both total attention and its composition. A workflow may be valuable even if total minutes are similar but a larger fraction of human attention shifts from routing to scientific judgement.

### 19.7 Quality and safety outcomes

Record end-to-end success, hard-constraint satisfaction, unsupported completion, false acceptance, failure-propagation size, recovery locality, detection latency, rework, reproducibility, citation validity, and claim/evidence mismatch. Resource metrics include model calls, tokens, wall time, verifier calls, tool calls, and peak compute/memory where relevant.

### 19.8 Human-attention validity threats

Attention measurement is vulnerable to expertise, familiarity, multitasking, interruption, and project-specific milestone weights. Use within-subject or matched-expertise designs where possible;

log only task-relevant active review time; freeze milestone weights before condition outcomes are visible; report raw intervention categories alongside any scalar attention score; and avoid interpreting $\rho_H$ as a universal cognitive-efficiency constant.

### 19.9 Analysis

The primary analysis should estimate condition-by-effective-horizon interactions using paired instances and mixed-effects models, with confidence intervals and transparent missing/failure handling. The key prediction is not necessarily that C3 dominates on short tasks. It is that relative benefit should grow when dependency depth, delayed faults, and coordination burden make flat execution brittle. Human-attention claims require both equal-or-better verified progress and lower or better-allocated supervisory attention under the frozen measurement protocol.

### 19.10 Current evidence boundary

Current conformance tests and deterministic fixtures validate runtime semantics only. The comparative study remains planned. No paper or report should state that the workflow has already reduced human-attention intensity or outperformed flat agents until frozen comparative evidence exists.

## 20 Discussion

### 20.1 A formalized version of what researchers already do

The most realistic comparison is not "workflow versus no workflow." Researchers already combine chat, research, notebooks, code agents, scripts, and paper tools. AI Native Workflow externalizes the coordination logic that otherwise remains in personal habits and conversation history. Its value should therefore be judged partly by whether it makes existing good practice easier to reproduce, inspect, and hand off.

### 20.2 Human attention as the scarce control resource

As agent capability increases, the amount of machine work that can be produced between human interventions grows. Without better state and interfaces, the supervisory burden can grow as well: more branches, more logs, more potential inconsistencies. The architecture attempts to decouple these quantities. Machine activity may grow while the human-facing DIC and handoff remain bounded in size and centered on material decisions.

### 20.3 Why hierarchy is useful but not automatically beneficial

Hierarchy introduces overhead: more artifacts, more integration, more opportunities for metadata errors. It is justified when the task contains enough dependency depth, uncertainty, parallelizable work, or evidence sensitivity that a flat interaction would otherwise require substantial human coordination. Short, local, reversible tasks should remain simple.

### 20.4 Scientific ambition and defensive drift

Evidence discipline should constrain scientific ambition, not erase it. A workflow that repeatedly asks hostile reviewers to remove every risky statement can converge toward trivial but defensible results. Primary Aim and salience controls are intended to preserve the stronger optimization problem: seek the best supported contribution within hard correctness and authority constraints.

### 20.5 DIC quality as a bottleneck

A poor DIC can shift rather than reduce burden. If it is verbose, ambiguous, or constantly rewritten, the researcher may spend more attention interpreting the governance artifact than supervising the science. Future evaluation should therefore measure DIC comprehension, edit/review time, disagreement between readers, and the amount of raw execution context humans still need to open.

### 20.6 Relation to software-engineering control structures

Many mechanisms have analogues in build graphs, contracts, runtime verification, checkpointing, and workflow engines. That is a strength and a limitation. The framework should not market old systems ideas as new mathematics. The research question is whether integrating them with modern language-agent surfaces changes long-horizon reliability and human-supervision economics enough to justify the operational complexity.

### 20.7 Generalization beyond research

The same pattern applies to engineering, quantitative analysis, documentation, web applications, and multi-artifact production. Research adds particularly strong requirements around evidence classes, frozen protocols, novelty, and claim boundaries, which is why the current educational/publication framing emphasizes researchers. Small non-research maintenance may use only a narrow subset of the runtime.

### 20.8 Future development

Priority directions include executing the matched-budget evaluation, improving automatic dependency discovery without hiding uncertainty, measuring DIC comprehension and attention burden, studying verifier diversity/correlation, testing model-family routing, and learning which scaffolding becomes redundant as agent surfaces improve. The workflow should be willing to delete controls whose empirical value disappears.

## 21 Limitations

AI Native Workflow adds explicit state and may be unnecessary overhead for short tasks. Human-attention reduction is not established; verbose or poorly prioritized DICs may shift coordination burden rather than reduce it. The attention metric depends on project-specific progress weights and imperfect proxies for cognitive effort. Read/write footprints can be incomplete, invalidating local-recovery guarantees. Evidence can be stale, adversarial, or weakly related to a claim. Verifiers can share correlated blind spots. Role specialization does not create epistemic independence. Parallel execution can increase total inference and introduce integration failure. External services, credentials, network state, and destructive actions require security controls beyond the workflow runtime. The formal checkpoint model assumes simplified stationarity. The current comparative benchmark is planned rather than confirmatory. Model/provider interfaces and official prompting guidance are time-sensitive and require periodic source-lock refresh.

The report is intentionally detailed and therefore contains more explanatory material than should appear in ordinary model prompts. Loading the entire report or detailed guide into every Worker would contradict the context-locality design. Runtime agents should read compact role docs and task-local contracts; humans/maintainers open the long-form material when explanation, maintenance, or training requires it.

# 22 Conclusion

AI Native Workflow 5.6.39 formalizes the multi-surface AI workflows researchers increasingly assemble from persistent chat, research systems, coding agents, files, tests, and manual coordination. It does not attempt to replace scientific judgement or claim that hierarchy is novel. It makes the hierarchy explicit and durable: Goal and Plan preserve programme intent; each Phase has one human-intelligible DIC; `PLAN.md` records the durable dependency and parallelism graph; Main materializes that graph into the initial EPS/prompt stack, while the coding-agent Orchestrator model-tunes and executes it and may issue bounded repair EPS revisions; evidence-gated verification chooses accept, repair, replan, or escalate; and package state allows recovery without reconstructing the project from chat history.

The practical target is a change in the allocation of attention. Researchers should spend less attention on routine routing, context transfer, status reconstruction, stale-file inference, and local repair, and more on the Primary Aim, surprising evidence, methodology, claim boundaries, and external responsibility. Whether the current implementation achieves that target is an empirical question. The next scientific step is therefore controlled measurement under matched task information and budgets, with both end-to-end research quality and human-attention intensity treated as first-class outcomes.

# A Operational Checklist

## A.1 Before a Phase

- Recover repository/package state rather than relying on memory.
- Confirm active Goal/Plan and the proposed Phase Primary Aim.
- Decide whether Research, Theory, or Experiment support is materially useful.
- Create exactly one DIC with Core README/REQUESTS/PLAN views.
- Check that primary versus optional requests are distinguishable.
- Confirm evidence classes and human/material escalation boundaries.

## A.2 Before delegation

- Confirm DIC plan dependencies and safe parallel groups.
- Give each executor one bounded primary outcome and clear return evidence.
- Resolve model/surface and prompt-tuning overlay.
- Declare read/write ownership for parallel siblings.
- Name the parent integration/merge node.

## A.3 Before a destructive, external, paid, or public action

- Verify exact target identity and authority.
- Use least privilege and isolate credentials.
- Prefer dry run/snapshot/rollback where feasible.
- Obtain explicit human approval when the action crosses the declared boundary.
- Preserve post-action evidence.

## A.4 Before claiming Phase completion

- Every primary request has current evidence or a visible terminal negative/block state.

- Verifier outcome is `accept` for closure.
- Failed and superseded branches remain recoverable.
- Current DIC and terminal EPS/run identities resolve.
- Human-facing handoff states the Primary Aim, decisive evidence, caveats, and any next material decision.

# B Example Multi-View DIC

### README.md skeleton

```
# Phase README
Primary Aim: <one central result>
Why now: <programme context>
Established evidence: <compact pointers>
Supporting questions: <ranked>
Constraints: <hard boundaries>
Optional branches: <drop if non-load-bearing>
Non-goals: <explicit exclusions>
Human/material gates: <exact>
Read order: <files>
```

### REQUESTS.md skeleton

```
# Requests
P1 PRIMARY -- <observable outcome>
  Acceptance: <predicate>
  Evidence class: <locked/proof/verified/etc>

S1 SUPPORTING -- <outcome>
  Acceptance: ...

O1 OPTIONAL -- <branch>
  Continue only if it strengthens/falsifies Primary Aim.
```

### PLAN.md skeleton

```
# Durable Phase Plan
A research / novelty  --\
B theory / audit       ---+--> D Main reconciliation --> E final evidence gate
C experiment / control --/

Dependencies: ...
Parallel group: A,B,C when interfaces permit
Merge node: D
Evidence gate: E
Repair branches: local execution defect -> fresh EPS
Replan trigger: method/acceptance/Primary Aim change -> Main
```

# C Example Bounded Executor Contract

```
Role: bounded repository executor
Primary outcome: implement the frozen experiment runner for DIC node C
Read first: DIC/README.md, DIC/REQUESTS.md, DIC/PLAN.md, protocol.md
Allowed writes: src/experiment/**, tests/experiment/**, artifacts/C/**
Forbidden: paper claims, protocol changes, release/submission
Evidence: tests, run manifest, exact changed files, failure record
Stop: if protocol is inconsistent, credentials/paid compute are required,
      or implementation requires changing the estimand
Return: concise result + evidence paths + unresolved blockers
```

# D Example Verifier Contract

A verifier receives the acceptance-bearing requests, relevant artifacts, evidence pointers, and provenance required to decide them. It should not inherit implementation authority. Its return is a structured list of accepted request IDs, failed predicates, evidence gaps, and one of accept/repair/replan/escalate. It must distinguish absence of evidence from evidence of failure.

# E Human-Attention Measurement Record

A candidate intervention record includes:

```
project_id
phase_id
timestamp_start / timestamp_end
intervention_class
  semantic_clarification | scientific_decision | manual_routing |
  context_reconstruction | execution_troubleshooting |
  evidence_interpretation | claim_release | security_external
trigger
materiality
human_action
changed_DIC_or_plan: true/false
verified_progress_milestone
notes
```

The measurement protocol should avoid intrusive logging that itself materially changes the attention being measured.

# F Tracking-File Authority Map

| File | Authority/purpose |
|---|---|
| `00_READ_FIRST.md` | shortest current authority/index |
| `CURRENT_STATE.md` | canonical human-readable truth now |
| `ROADMAP.md` | future work only |
| `README.md` | stable package explanation |
| `QUICKSTART.md` | adoption path only |
| `VERSIONING.md` | version policy |
| `CHANGELOG.md` | cumulative history |
| `PACKAGE_MANIFEST.json` | file inventory/hashes |
| `PACKAGE_HANDOFF.json` | machine-readable recovery state |
| `history/` | superseded reports, duplicate quick starts, old changelogs, migration notes |

**Table 7:** Development-distribution tracking authorities.

# References

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.

[2] T. Brown et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, 2020.

[3] J. Hoffmann et al. Training compute-optimal large language models. *arXiv:2203.15556*, 2022.

[4] G. Hinton, O. Vinyals, and J. Dean. Distilling the knowledge in a neural network. *arXiv:1503.02531*, 2015.

[5] L. Ouyang et al. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, 2022.

[6] OpenAI. Aligning language models to follow instructions. Technical publication, 2022. https://openai.com/index/instruction-following/

[7] OpenAI. Introducing ChatGPT. Product and methods description, 2022. https://openai.com/index/chatgpt/

[8] OpenAI. Learning to summarize with human feedback. Research publication, 2020. https://openai.com/index/learning-to-summarize-with-human-feedback/

[9] R. Rafailov, A. Sharma, E. Mitchell, C. D. Manning, S. Ermon, and C. Finn. Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems*, 2023.

[10] N. Shazeer. Fast Transformer decoding: One write-head is all you need. *arXiv:1911.02150*, 2019.

[11] J. Ainslie et al. GQA: Training generalized multi-query Transformer models from multi-head checkpoints. In *Empirical Methods in Natural Language Processing*, 2023.

[12] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*, 2022.

[13] W. Kwon et al. Efficient memory management for large language model serving with PagedAttention. In *ACM Symposium on Operating Systems Principles*, 2023.

[14] N. F. Liu et al. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024.

[15] S. Yao et al. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.

[16] J. Yang et al. SWE-agent: Agent–computer interfaces enable automated software engineering. *arXiv:2405.15793*, 2024.

[17] B. Z. Li, A. Tamkin, N. D. Goodman, and J. Andreas. Eliciting human preferences with language models. In *International Conference on Learning Representations*, 2025.

[18] K. Kobalczyk, N. Astorga, T. Liu, and M. van der Schaar. Active task disambiguation with LLMs. In *International Conference on Learning Representations*, 2025.

[19] M. J. Q. Zhang, W. B. Knox, and E. Choi. Modeling future conversation turns to teach LLMs to ask clarifying questions. *arXiv:2410.13788*, 2024.

[20] M. Sharma et al. Towards understanding sycophancy in language models. *arXiv:2310.13548*, 2023.

[21] M. Steyvers et al. What large language models know and what people think they know. *Nature Machine Intelligence*, 7:221–231, 2025. doi:10.1038/s42256-024-00976-7.

[22] W. Laurito, B. Davis, P. Grietzer, T. Gavenčiak, A. Böhm, and J. Kulveit. AI–AI bias: Large language models favor communications generated by large language models. *Proceedings of the National Academy of Sciences*, 122, 2025. doi:10.1073/pnas.2415697122.

[23] Y. Zhou et al. Large language models are human-level prompt engineers. *arXiv:2211.01910*, 2022.

[24] Y. Wang et al. Self-Instruct: Aligning language models with self-generated instructions. In *Annual Meeting of the Association for Computational Linguistics*, 2023.

[25] C. Yang et al. Large language models as optimizers. *arXiv:2309.03409*, 2023.

[26] Y. Du, M. Tian, S. Ronanki, S. Rongali, S. B. Bodapati, A. Galstyan, A. Wells, R. Schwartz, E. A. Huerta, and H. Peng. Context length alone hurts LLM performance despite perfect retrieval. In *Findings of the Association for Computational Linguistics: EMNLP*, pages 23281–23298, 2025. doi:10.18653/v1/2025.findings-emnlp.1264.

[27] P. Laban, H. Hayashi, Y. Zhou, and J. Neville. LLMs get lost in multi-turn conversation. *arXiv:2505.06120*, 2025.

[28] M. Zheng, J. Pei, L. Logeswaran, M. Lee, and D. Jurgens. When "a helpful assistant" is not really helpful: Personas in system prompts do not improve performances of large language models. In *Findings of the Association for Computational Linguistics: EMNLP*, pages 15126–15154, 2024. doi:10.18653/v1/2024.findings-emnlp.888.

[29] P. H. Luz de Araujo, P. Röttger, D. Hovy, and B. Roth. Principled personas: Defining and measuring the intended effects of persona prompting on task performance. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 26857–26886, 2025. doi:10.18653/v1/2025.emnlp-main.1364.

[30] J. Wei, D. Huang, Y. Lu, D. Zhou, and Q. V. Le. Simple synthetic data reduces sycophancy in large language models. *arXiv:2308.03958*, 2023.

[31] T. Zhang, J. Yuan, and S. Avestimehr. Revisiting OPRO: The limitations of small-scale LLMs as optimizers. In *Findings of the Association for Computational Linguistics: ACL*, 2024; *arXiv:2405.10276*.

[32] T. Everitt, C. Garbacea, A. Bellot, J. Richens, H. Papadatos, S. Campos, and R. Shah. Evaluating the goal-directedness of large language models. *arXiv:2504.11844*, 2025.

[33] W. Hou, L. Zhou, H.-Y. Hu, Y. Chen, Y.-Z. You, and X.-L. Qi. How focused are LLMs? A quantitative study via repetitive deterministic prediction tasks. *arXiv:2511.00763v2*, 2025.

[34] N. Sardana, J. Portes, S. Doubov, and J. Frankle. Beyond Chinchilla-optimal: Accounting for inference in language model scaling laws. *arXiv:2401.00448*, 2024.

[35] L. Twist, M. Harman, D. Syme, J. Noppen, H. Yannakoudakis, D. Nauck, and J. M. Zhang. A study of LLMs' preferences for libraries and programming languages. Accepted to Findings of the Association for Computational Linguistics: ACL 2026; *arXiv:2503.17181v4*.

[36] V. Tagliabue and L. Dung. Probing the preferences of a language model: Integrating verbal and behavioral tests of AI welfare. *arXiv:2509.07961*, 2025.

[37] O. Gilg, P. Beckmann, D. Paleka, and P. Butlin. Probing persona-dependent preferences in language models. *arXiv:2605.13339*, 2026.

[38] C. Pipis, S. Garg, V. Kontonis, V. Shrivastava, A. Krishnamurthy, and D. Papailiopoulos. Wait, wait, wait... why do reasoning models loop? *arXiv:2512.12895*, 2025.

[39] Y. Li, X. Yang, L. Wang, W. Luo, and H. Chen. ComplexMCP: Evaluation of LLM agents in dynamic, interdependent, and large-scale tool sandbox. *arXiv:2605.10787*, 2026.

[40] R. Marchand, A. O Cathain, J. Wynne, P. M. Giavridis, S. Deverett, J. Wilkinson, J. Gwartz, and H. Coppock. Quantifying frontier LLM capabilities for container sandbox escape. *arXiv:2603.02277*, 2026.

[41] OpenAI. Running Codex safely at OpenAI. Security practice note, 2026. https://openai.com/index/running-codex-safely/

[42] OpenAI. GPT-5.6: Frontier intelligence that scales with your ambition. Product release, 9 July 2026. https://openai.com/index/gpt-5-6/

[43] OpenAI. GPT-5.6 System Card. Deployment Safety Hub, 9 July 2026. https://deploymentsafety.openai.com/gpt-5-6

[44] A. Panickssery, S. R. Bowman, and S. Feng. LLM evaluators recognize and favor their own generations. In *Advances in Neural Information Processing Systems*, 2024; *arXiv:2404.13076*.

[45] Y. Choi, C. Li, Y. Yang, and Z. Jin. Agent-to-agent theory of mind: Testing interlocutor awareness among large language models. *arXiv:2506.22957*, 2025.

[46] L. Zhang, J. Wang, F. Xue, and Y.-Y. Chu. Post-training recipe, more than model family, shapes multi-agent LLM conversational behavior. *arXiv:2606.20632*, 2026.

[47] A. Yang et al. Qwen3 technical report. *arXiv:2505.09388*, 2025.

[48] Anthropic. Anthropic's Transparency Hub: model reports. Technical documentation, updated 2026. https://www.anthropic.com/transparency/model-report

[49] DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv:2501.12948*, 2025. https://github.com/deepseek-ai/DeepSeek-R1

[50] Google DeepMind. Gemini family updates: Gemini 1.5 Flash and distillation from Gemini 1.5 Pro. Product and technical announcement, 14 May 2024. https://blog.google/innovation-and-ai/products/google-gemini-update-flash-ai-assistant-io-2024/

[51] Meta AI. Llama 3.2: edge models, structured pruning, and knowledge distillation. Technical announcement, 25 September 2024. https://ai.meta.com/blog/llama-3-2-connect-2024-vision-edge-mobile-devices/

[52] Anthropic. Claude Code CLI reference. Product documentation, accessed 30 July 2026. https://docs.anthropic.com/en/docs/claude-code/cli-usage

[53] Anthropic. Claude Code security and permission architecture. Product documentation, accessed 30 July 2026. https://docs.anthropic.com/en/docs/claude-code/security

[54] GitHub. Allowing and denying tool use in GitHub Copilot CLI. Product documentation, accessed 30 July 2026. https://docs.github.com/en/copilot/how-tos/copilot-cli/use-copilot-cli/allowing-tools

[55] Anthropic Applied AI Team. Effective context engineering for AI agents. Engineering note, 29 September 2025. https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents

[56] OpenAI. Harness engineering: Leveraging Codex in an agent-first world. Engineering note, 11 February 2026. https://openai.com/index/harness-engineering/

[57] Y. Zhang, R. Sun, Y. Chen, T. Pfister, R. Zhang, and S. O. Arik. Chain of agents: Large language models collaborating on long-context tasks. *arXiv:2406.02818*, 2024.

[58] Anthropic. How we built our multi-agent research system. Engineering report, 13 June 2025. https://www.anthropic.com/engineering/multi-agent-research-system

[59] OpenAI. The next evolution of the Agents SDK. Product and engineering note, 15 April 2026. https://openai.com/index/the-next-evolution-of-the-agents-sdk/

[60] Anthropic. Demystifying evals for AI agents. Engineering note, 9 January 2026. https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents

[61] OpenAI. Introducing deep research. Product and methods description, 2 February 2025, updated 10 February 2026. https://openai.com/index/introducing-deep-research/

[62] J. Wei et al. BrowseComp: A benchmark for browsing agents. OpenAI research publication, 10 April 2025. https://openai.com/index/browsecomp/

[63] OpenAI. Introducing Codex. Product and methods description, 16 May 2025. https://openai.com/index/introducing-codex/

[64] OpenAI. Introducing upgrades to Codex. Product and model release, 15 September 2025. https://openai.com/index/introducing-upgrades-to-codex/

[65] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan. $\tau$-bench: A benchmark for tool–agent–user interaction in real-world domains. *arXiv:2406.12045*, 2024.

[66] K. Zhu et al. Where LLM agents fail and how they can learn from failures. *arXiv:2509.25370*, 2025.

[67] K. Song et al. Aegis: Taxonomy and optimizations for overcoming agent–environment failures in LLM agents. *arXiv:2508.19504*, 2025.

[68] RTK contributors. RTK: Rust Token Killer, a CLI proxy for filtering agent terminal output. Open-source project documentation, accessed 30 July 2026. https://github.com/rtk-ai/rtk

[69] Stanford University. Language Models, Autumn 2025 lecture materials. https://stanford-cs221.github.io/autumn2025-lectures/language_models.pdf

[70] E. Brunskill. Lecture 1: Introduction to Reinforcement Learning, Winter 2026. Stanford University lecture materials. https://web.stanford.edu/class/cs234/slides/lecture1pre.pdf

[71] Stanford University. Language Modeling from Scratch, course materials. https://stanford-cs336.github.io/

[72] Yao, Shunyu and Zhao, Jeffrey and Yu, Dian and Du, Nan and Shafran, Izhak and Narasimhan, Karthik and Cao, Yuan. ReAct: Synergizing Reasoning and Acting in Language Models. International Conference on Learning Representations, 2023.

[73] Yao, Shunyu and Yu, Dian and Zhao, Jeffrey and Shafran, Izhak and Griffiths, Thomas L. and Cao, Yuan and Narasimhan, Karthik. Tree of Thoughts: Deliberate Problem Solving with Large Language Models. Advances in Neural Information Processing Systems, 2023.

[74] Besta, Maciej and Blach, Nils and Kubicek, Ales and Gerstenberger, Robert and Podstawski, Michal and Gianinazzi, Lukas and Gajda, Joanna and Lehmann, Tomasz and Niewiadomski, Hubert and Hoefler, Torsten. Graph of Thoughts: Solving Elaborate Problems with Large Language Models. AAAI Conference on Artificial Intelligence, 2024.

[75] Xu, Binfeng and Peng, Zhendong and Lei, Bowen and Mukherjee, Subhabrata and Liu, Yuchen and Xu, Dongkuan. ReWOO: Decoupling Reasoning from Observations for Efficient Augmented Language Models. arXiv preprint arXiv:2305.18323, 2023.

[76] Prasad, Archiki and Koller, Alexander and Hartmann, Mareike and Clark, Peter and Sabharwal, Ashish and Khot, Tushar. ADaPT: As-Needed Decomposition and Planning with Language Models. Findings of the Association for Computational Linguistics: NAACL, 2024.

[77] Zhou, Andy and Yan, Kai and Shlapentokh-Rothman, Michal and Wang, Haohan and Wang, Yu-Xiong. Language Agent Tree Search Unifies Reasoning, Acting, and Planning in Language Models. International Conference on Machine Learning, 2024.

[78] Shinn, Noah and Cassano, Federico and Gopinath, Ashwin and Narasimhan, Karthik and Yao, Shunyu. Reflexion: Language Agents with Verbal Reinforcement Learning. Advances in Neural Information Processing Systems, 2023.

[79] Wu, Qingyun and Bansal, Gagan and Zhang, Jieyu and Wu, Yiran and Li, Beibin and Zhu, Erkang and Jiang, Li and Zhang, Xiaoyun and Zhang, Shaokun and Liu, Jiale et al.. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. arXiv preprint arXiv:2308.08155, 2023.

[80] Li, Guohao and Hammoud, Hasan Abed Al Kader and Itani, Hani and Khizbullin, Dmitrii and Ghanem, Bernard. CAMEL: Communicative Agents for Mind Exploration of Large Scale Language Model Society. Advances in Neural Information Processing Systems, 2023.

[81] Chen, Weize and Su, Yusheng and Zuo, Jingwei and Yang, Cheng and Yuan, Chenfei and Chan, Chi-Min and Yu, Heyang and Lu, Yaxi and Hung, Yi-Hsin and Qian, Chen et al.. AgentVerse: Facilitating Multi-Agent Collaboration and Exploring Emergent Behaviors. International Conference on Learning Representations, 2024.

[82] Hong, Sirui and Zhuge, Mingchen and Chen, Jonathan and Zheng, Xiawu and Cheng, Yuheng and Zhang, Ceyao and Wang, Jinlin and Wang, Zili and Yau, Steven Ka Shing and Lin, Zijuan et al.. MetaGPT: Meta Programming for Multi-Agent Collaborative Framework. International Conference on Learning Representations, 2024.

[83] Packer, Charles and Fang, Vivian and Patil, Shishir G. and Lin, Kevin and Wooders, Sarah and Gonzalez, Joseph E.. MemGPT: Towards LLMs as Operating Systems. arXiv preprint arXiv:2310.08560, 2023.

[84] Wang, Zhenhailong and Liu, Shaoguang and Chen, Yixin and Zheng, Zhaoran and Wang, Mingyuan and Kawaguchi, Kenji and Yao, Huaxiu. HiAgent: Hierarchical Working Memory Management for Solving Long-Horizon Agent Tasks with Large Language Models. arXiv preprint, 2024.

[85] Liu, Xiao and Yu, Hao and Zhang, Hanchen and Xu, Yifan and Lei, Xuanyu and Lai, Hanyu and Gu, Yu and Ding, Hangliang and Men, Kaiwen and Yang, Kejuan et al.. AgentBench: Evaluating LLMs as Agents. International Conference on Learning Representations, 2024.

[86] Zhou, Shuyan and Xu, Frank F. and Zhu, Hao and Zhou, Xuhui and Lo, Robert and Sridhar, Abishek and Cheng, Xianyi and Bisk, Yonatan and Fried, Daniel and Alon, Uri and Neubig, Graham. WebArena: A Realistic Web Environment for Building Autonomous Agents. International Conference on Learning Representations, 2024.

[87] Mialon, Grégoire and Fourrier, Clémentine and Swift, Craig and Wolf, Thomas and LeCun, Yann and Scialom, Thomas. GAIA: A Benchmark for General AI Assistants. International Conference on Learning Representations, 2024.

[88] Jimenez, Carlos E. and Yang, John and Wettig, Alexander and Yao, Shunyu and Pei, Kexin and Press, Ofir and Narasimhan, Karthik. SWE-bench: Can Language Models Resolve Real-World GitHub Issues?. International Conference on Learning Representations, 2024.

[89] Xie, Tianbao and Zhang, Danyang and Chen, Jixuan and Li, Xiaochuan and Zhao, Siheng and Cao, Ruisheng and Hua, Tenyue and Cheng, Zhoujun and Shin, Dong-Suk and Lei, Fangyu et al.. OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. Advances in Neural Information Processing Systems, 2024.

[90] Khattab, Omar and Singhvi, Arnav and Maheshwari, Paridhi and Zhang, Zhiyuan and Santhanam, Keshav and Vardhamanan, Sri and Haq, Saiful and Sharma, Ashutosh and Joshi, Thomas T. and Moazam, Hanna et al.. DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines. International Conference on Learning Representations, 2024.

[91] Hu, Shengran and Lu, Cong and Clune, Jeff. Automated Design of Agentic Systems. International Conference on Learning Representations, 2025.

[92] Lightman, Hunter and Kosaraju, Vineet and Burda, Yura and Edwards, Harrison and Baker, Bowen and Lee, Teddy and Leike, Jan and Schulman, John and Sutskever, Ilya and Cobbe, Karl. Let's Verify Step by Step. arXiv preprint arXiv:2305.20050, 2023.

[93] Irving, Geoffrey and Christiano, Paul and Amodei, Dario. AI Safety via Debate. arXiv preprint arXiv:1805.00899, 2018.

[94] Leike, Jan and Krueger, David and Everitt, Tom and Martic, Miljan and Maini, Vishal and Legg, Shane. Scalable Agent Alignment via Reward Modeling: A Research Direction. arXiv preprint arXiv:1811.07871, 2018.

[95] Burns, Collin and Ye, Haotian and Klein, Dan and Steinhardt, Jacob. Weak-to-Strong Generalization: Eliciting Strong Capabilities with Weak Supervision. International Conference on Machine Learning, 2024.

[96] Ruan, Yangjun and Dong, Honghua and Wang, Andrew and Pitis, Silviu and Zhou, Yongchao and Ba, Jimmy and Dubois, Yann and Maddison, Chris J. and Hashimoto, Tatsunori. ToolEmu: Identifying the Risks of LM Agents with an LM-Emulated Sandbox. International Conference on Learning Representations, 2024.

[97] Debenedetti, Edoardo and Zhang, Jie and Balunovic, Mislav and Beurer-Kellner, Luca and Fischer, Marc and Tramèr, Florian. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. Advances in Neural Information Processing Systems, 2024.

[98] LeCun, Yann. A Path Towards Autonomous Machine Intelligence. OpenReview preprint, 2022.

[99] Bardes, Adrien and Garrido, Quentin and Ponce, Jean and Rabbat, Michael and LeCun, Yann and Assran, Mahmoud and Ballas, Nicolas. V-JEPA: Latent Video Prediction for Visual Representation Learning. International Conference on Machine Learning, 2024.

[100] Hoare, C. A. R.. An Axiomatic Basis for Computer Programming. Communications of the ACM, 1969.

[101] Ferrante, Jeanne and Ottenstein, Karl J. and Warren, Joe D.. Program Dependence Graphs and Their Use in Optimization. ACM Transactions on Programming Languages and Systems, 1987.

[102] Mokhov, Andrey and Mitchell, Neil and Peyton Jones, Simon. Build Systems à la Carte. Proceedings of the ACM on Programming Languages, 2018.

[103] Acar, Umut A.. Self-Adjusting Computation. Morgan & Claypool, 2009.

[104] Young, John W.. A First Order Approximation to the Optimum Checkpoint Interval. Communications of the ACM, 1974.

[105] Daly, John T.. A Higher Order Estimate of the Optimum Checkpoint Interval for Restart Dumps. Future Generation Computer Systems, 2006.

[106] Leucker, Martin and Schallhart, Christian. A Brief Account of Runtime Verification. Journal of Logic and Algebraic Programming, 2009.